ADVANCED ROBOTIC PERCEPTION / IMAGE PROCESSING AND COMPUTER VISION

CONVOLUTIONAL NEURAL NETWORKS

RIKKE GADE 2026

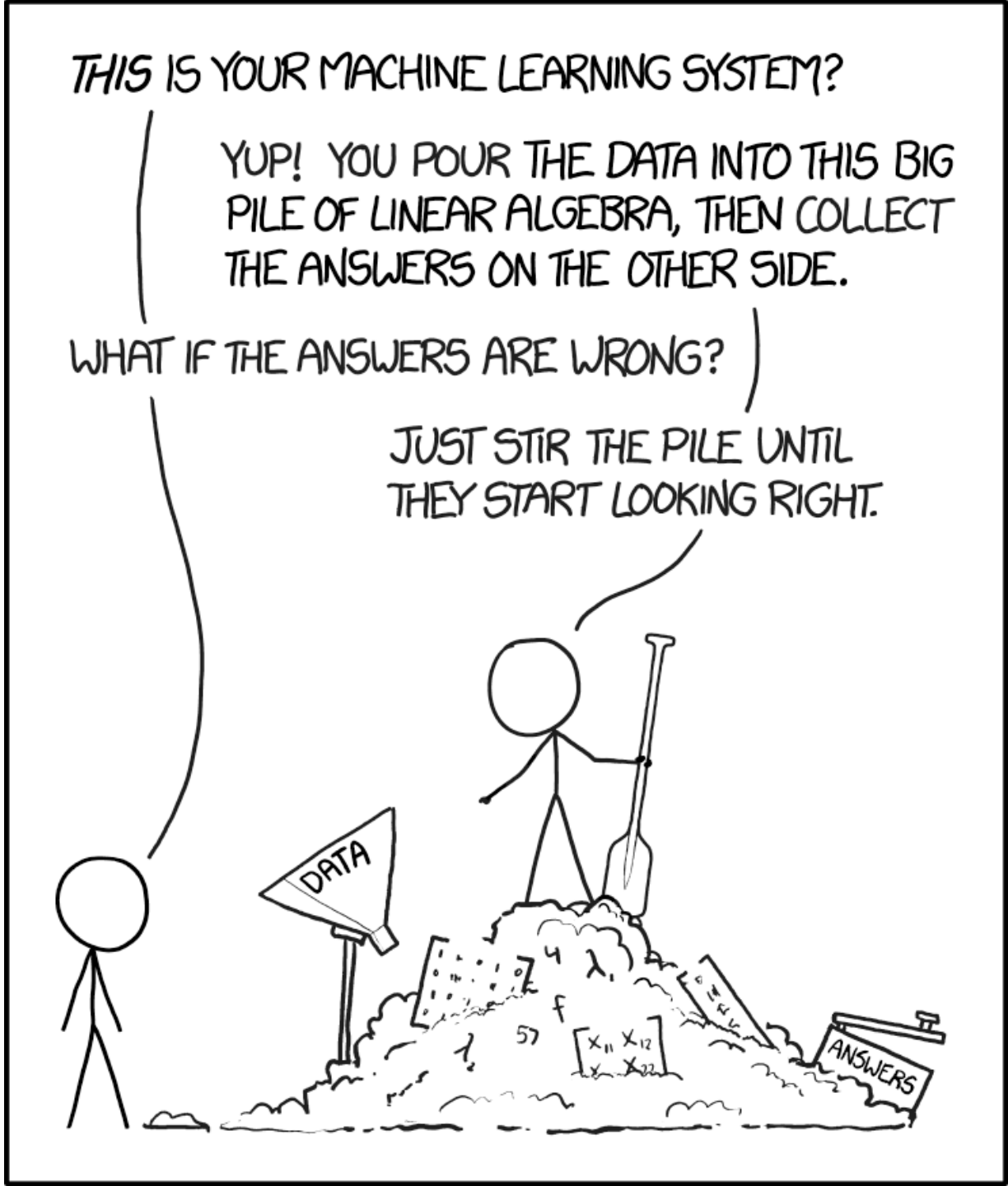
SLIDES BY CHRIS H. BAHNSEN



AAU CREATE UNIVERSITY
AALBORG UNIVERSITY
DENMARK

Agenda

- A primer on Neural Networks
- Convolutional Neural Networks



https://xkcd.com/1838/

Deep learning and Neural Networks

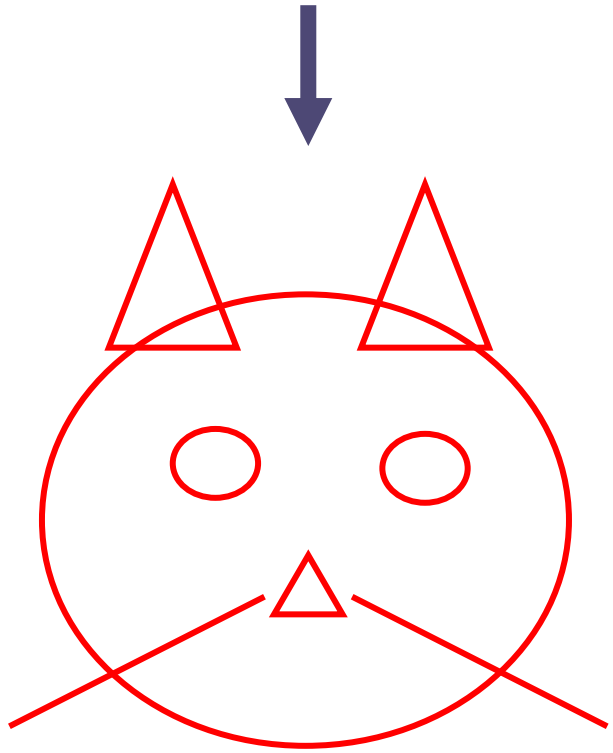


AALBORG UNIVERSITY
DENMARK

Handcrafted models

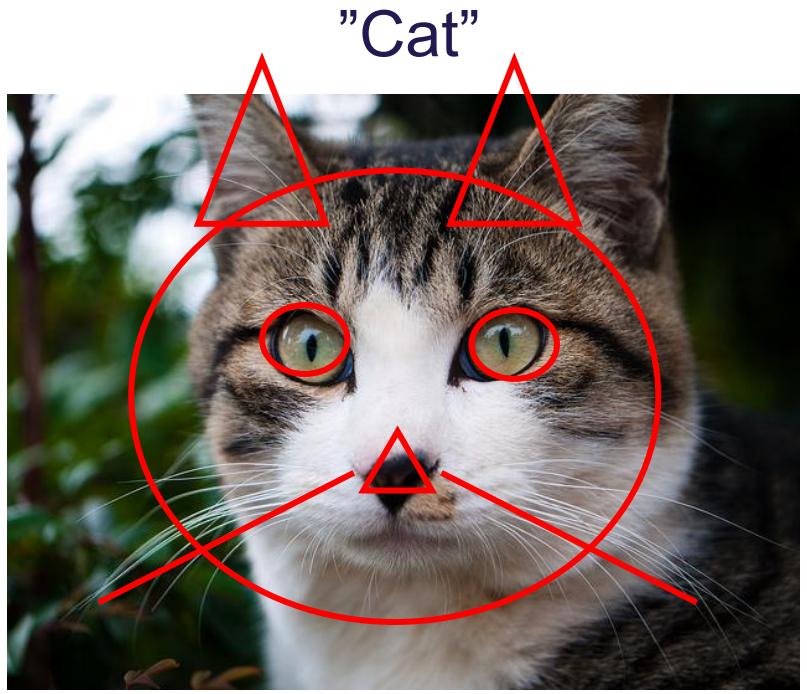
- Task: Find cat
- Solution: Rule-based part model

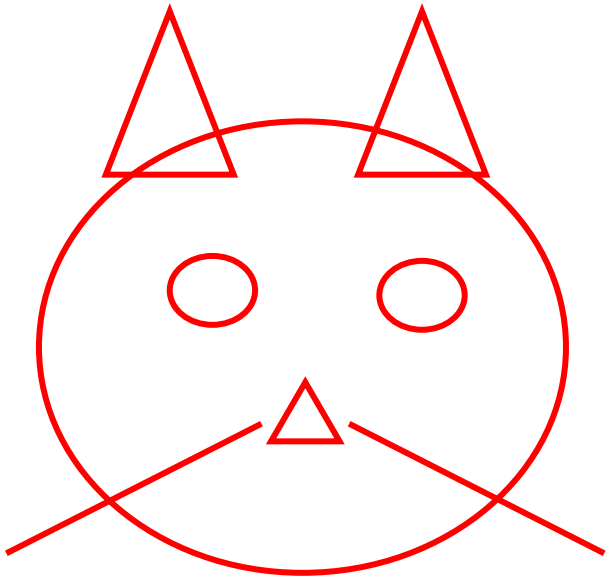




Handcrafted models

- Task: Find cat
- Solution: Rule-based part model

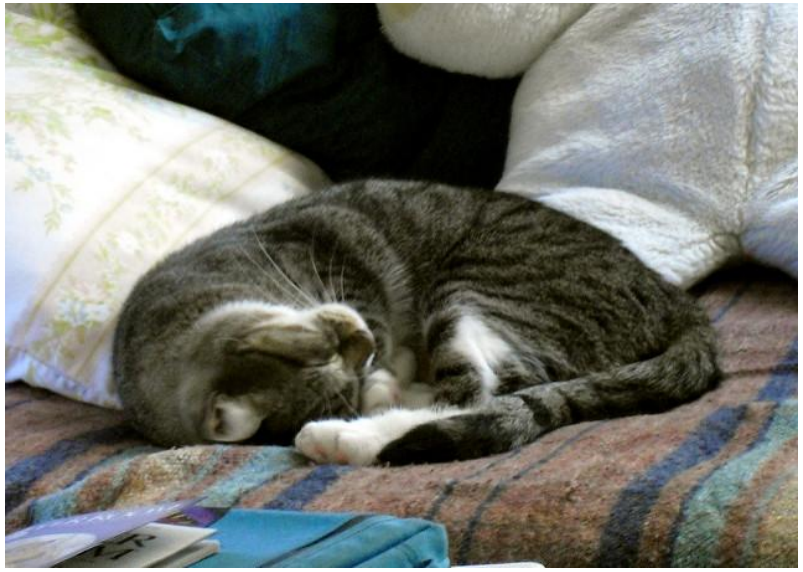


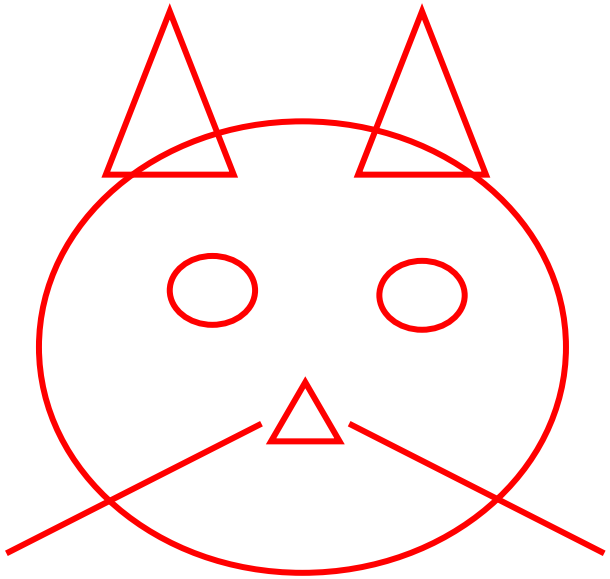


Handcrafted models

- Does not work with different appearances
- Cats are deformable

?





Handcrafted models

- Impossible to model all cat deformations by hand
- We want to *learn* our model instead of hard-coding it









Machine learning

- Handcrafted models
  - Handcrafted features
  - Rule-based classification



- Machine learning
  - Handcrafted features
  - Learned classification

Machine learning

- Example: Differentiate between Per and Lene
- Approach: find discriminative features

”Per”



”Lene”



Machine learning

- Example: Differentiate between Per and Lene
- Amount of hair?

”Per”



”Lene”

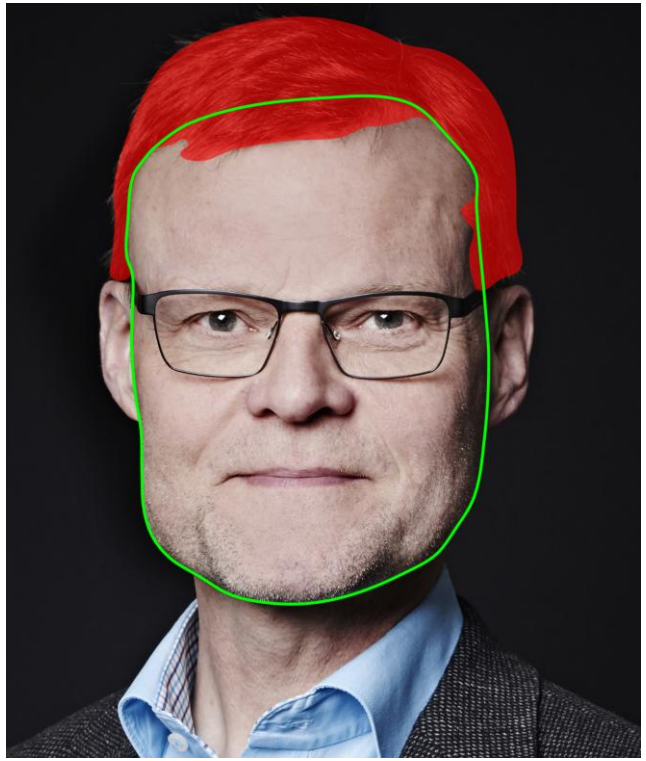


Hair-ratio	0,09	0,24

Machine learning

- Example: Differentiate between Per and Lene
- Amount of hair?
- Face circularity?

”Per”



”Lene”



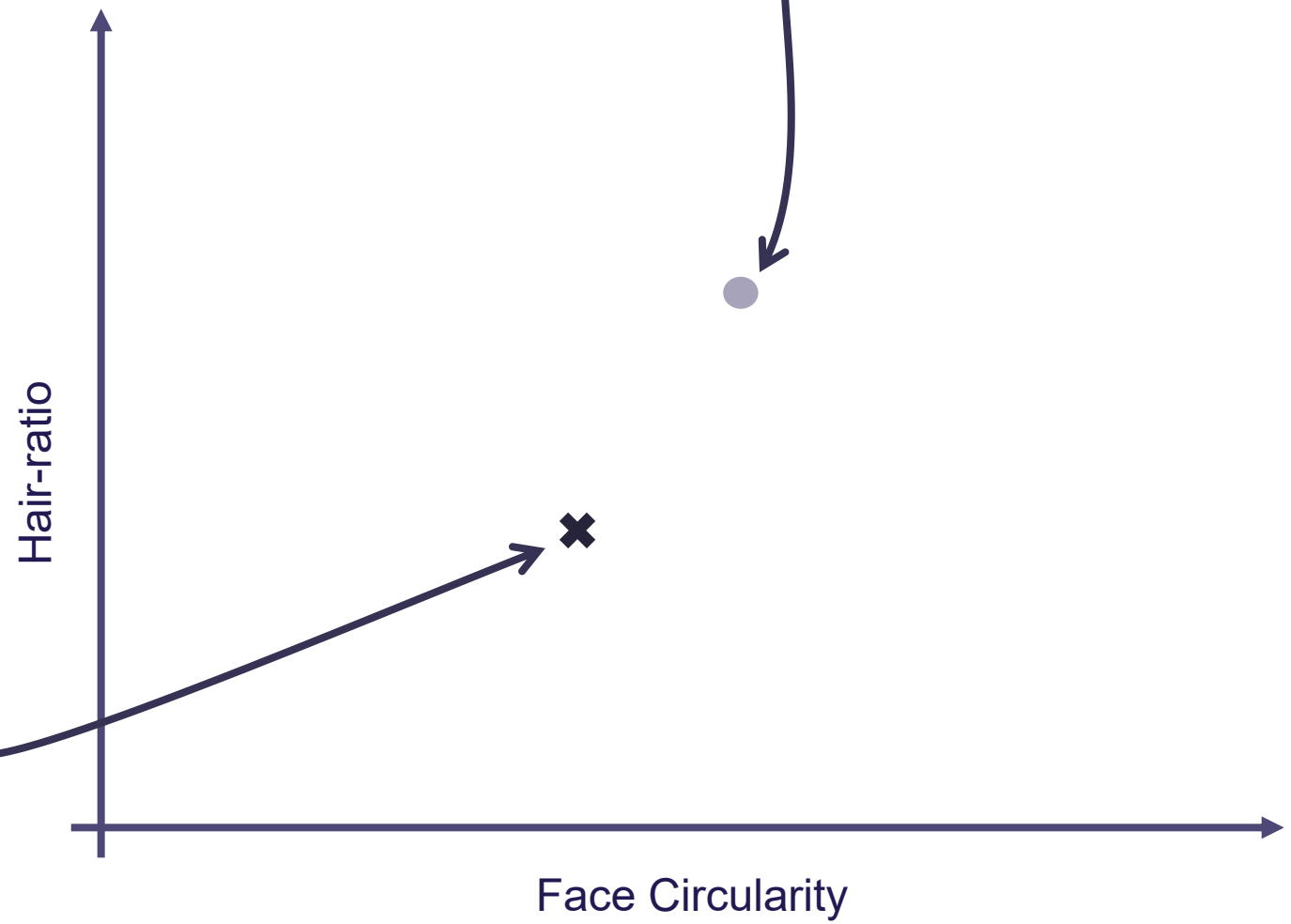
Hair-ratio	0,09	0,24
Circularity	0,79	0,87

Machine learning

- Example: Differentiate between Per and Lene
- Amount of hair?
- Face circularity?
- Feature-space





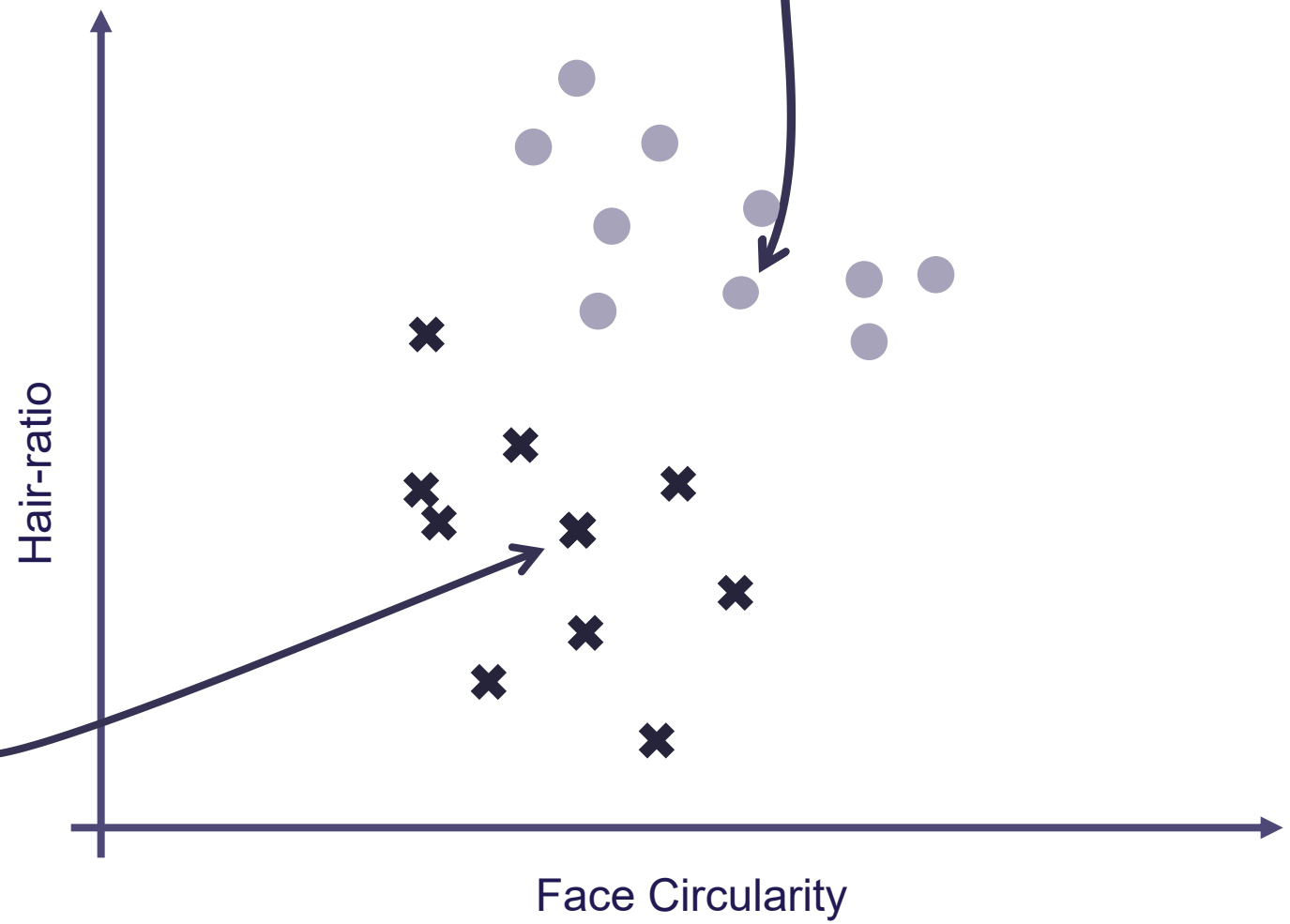


Machine learning

- Example: Differentiate between Per and Lene
- Amount of hair?
- Face circularity?
- Multiple images

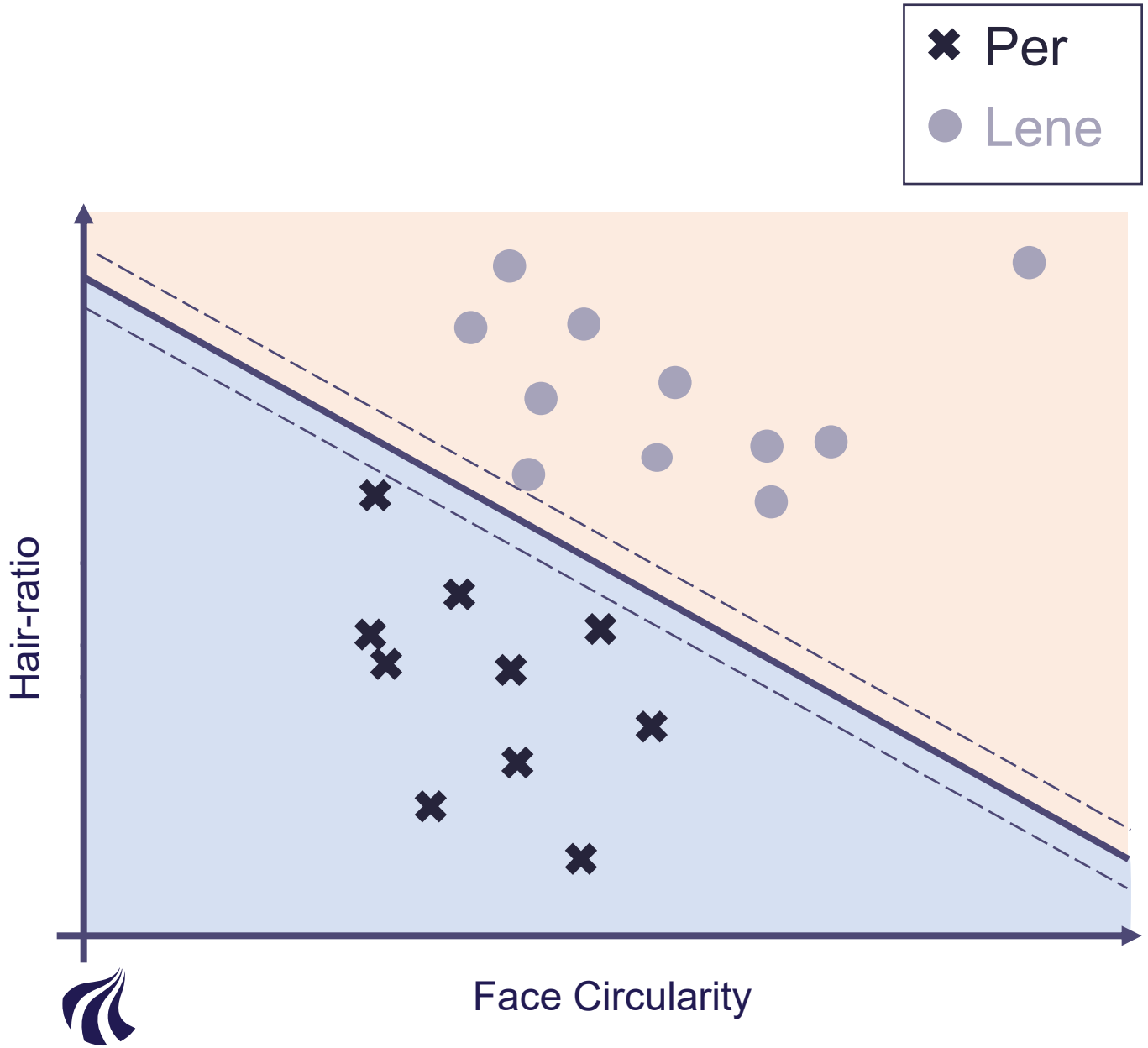






Machine learning

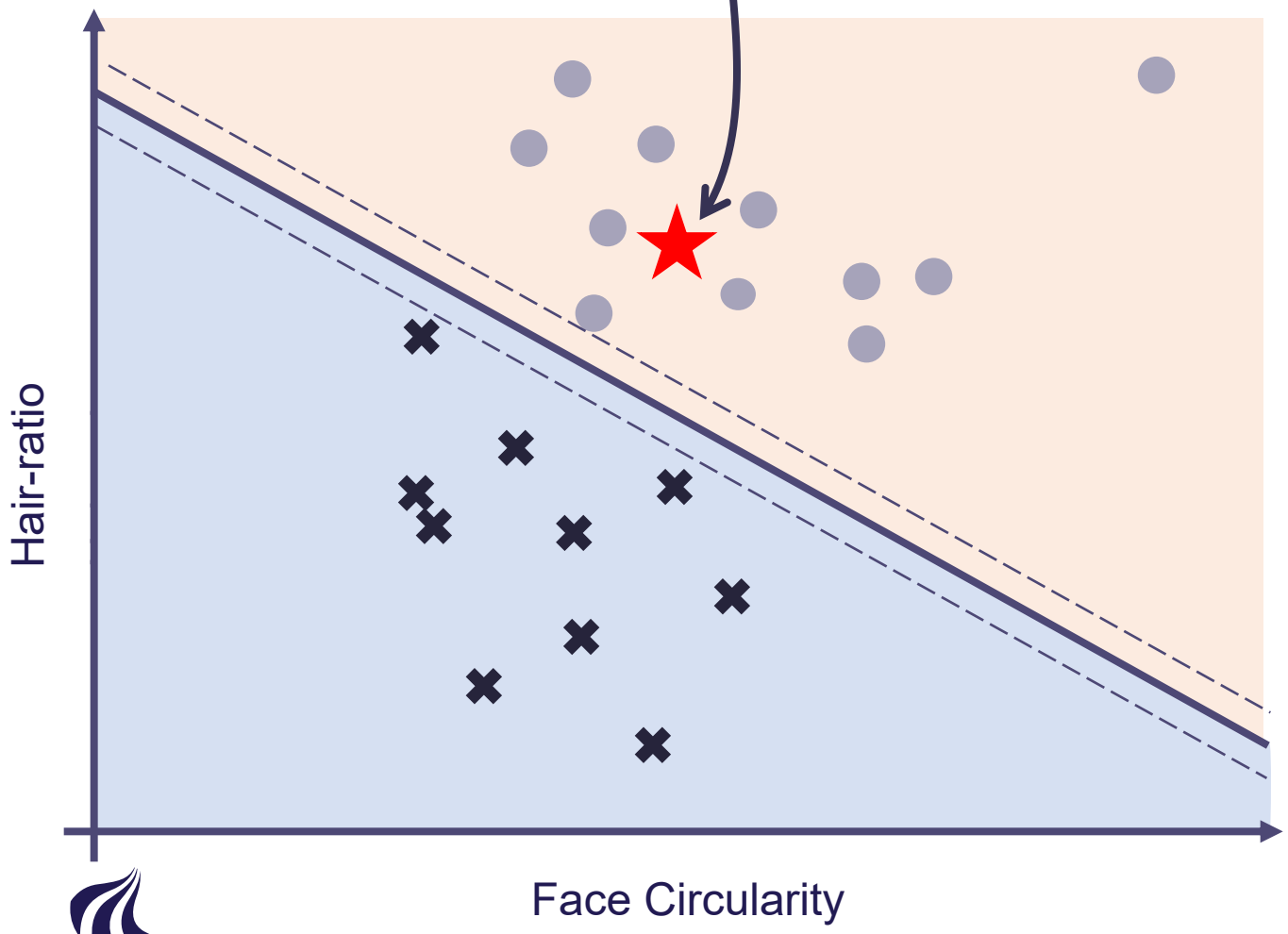
- Example: Differentiate between Per and Lene
- Amount of hair?
- Face circularity?
- Find line separating the two regions



Machine learning

- New image – who?
  - Extract features
  - Where in feature space?
  - Compare to line
  - Decide who it is
- This is machine learning





Deep learning

- Handcrafted models
  - Handcrafted features
  - Rule-based classification



- Machine learning
  - Handcrafted features
  - Learned classification



- Deep learning
  - Learned features
  - Learned classification
  - Handcrafted networks

Deep learning

- Deep learning
  - Learned features
  - Learned classification
  - Handcrafted networks

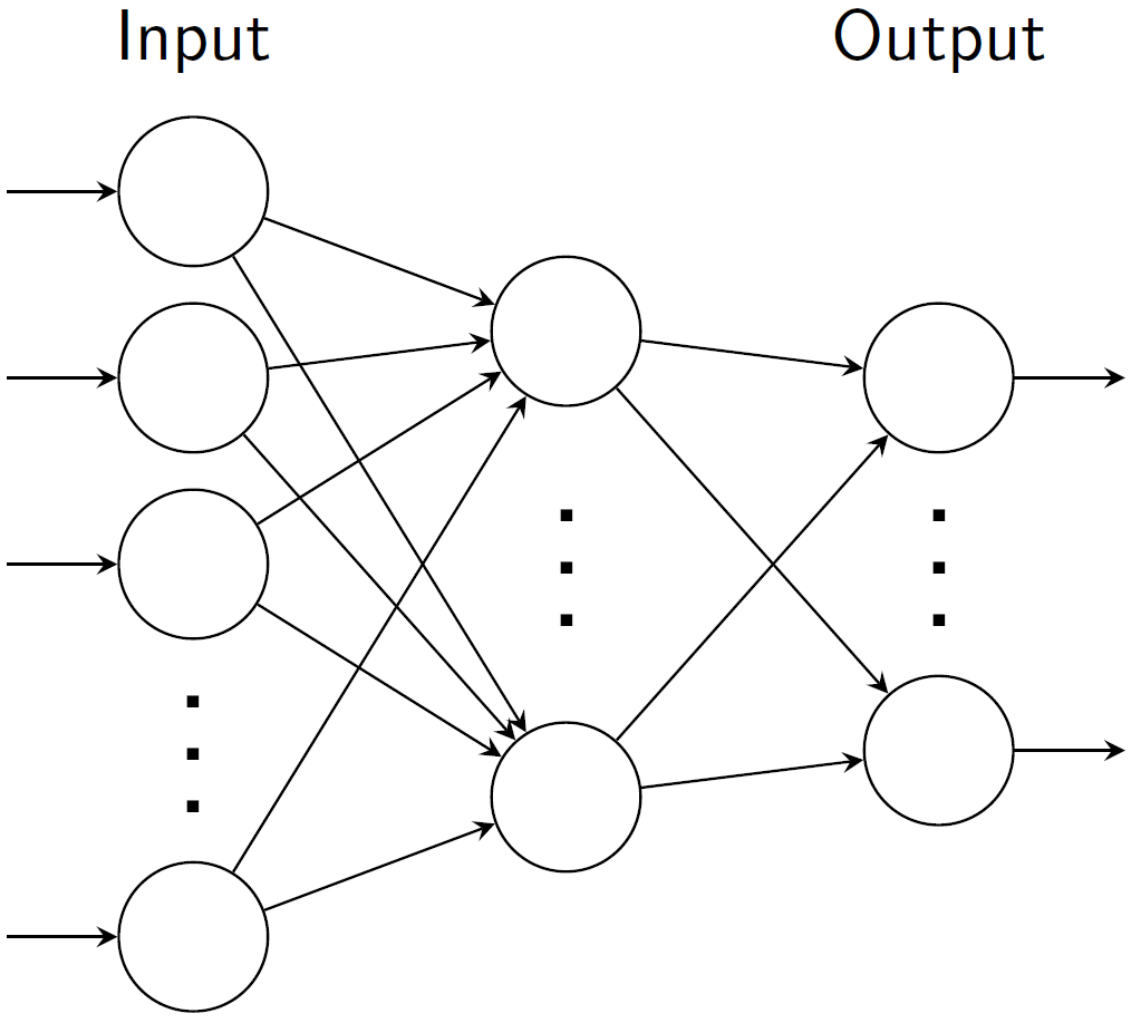


- Deep neural networks with generic structure, e.g. fully connected, do not work
- Architecture of network has to be appropriate to domain
  - For images, exploit 2D grid, local dependencies etc.

[Deep learning, LeCun, Bengio, Hinton, Nature 2015]

Neural Networks

- Neural networks composed of layers of artificial neurons
- Each layer computes sum function of layer beneath
- Inputs mapped in feed-forward fashion to output
- Only feed-forward neural models for this lecture



An individual neuron

- Input:
  x (n x 1 vector)
- Parameters:
  weights *w* (n vector), bias *b* (scalar)
- Activation:

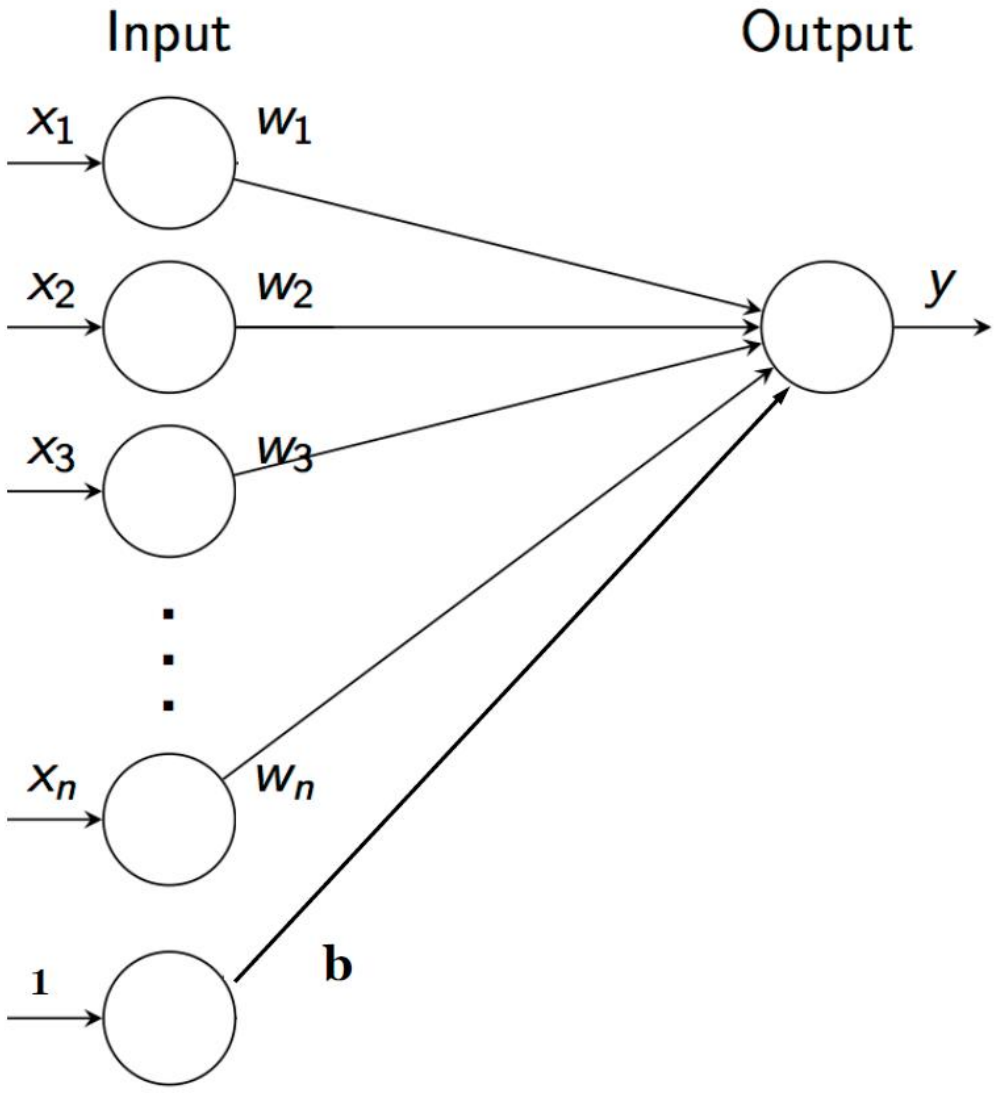
$$a = \sum_{i=1}^{n} x_i w_i + b$$

- Point-wise non-linear function, e.g.:

$$\sigma(.) = \tanh(.)$$

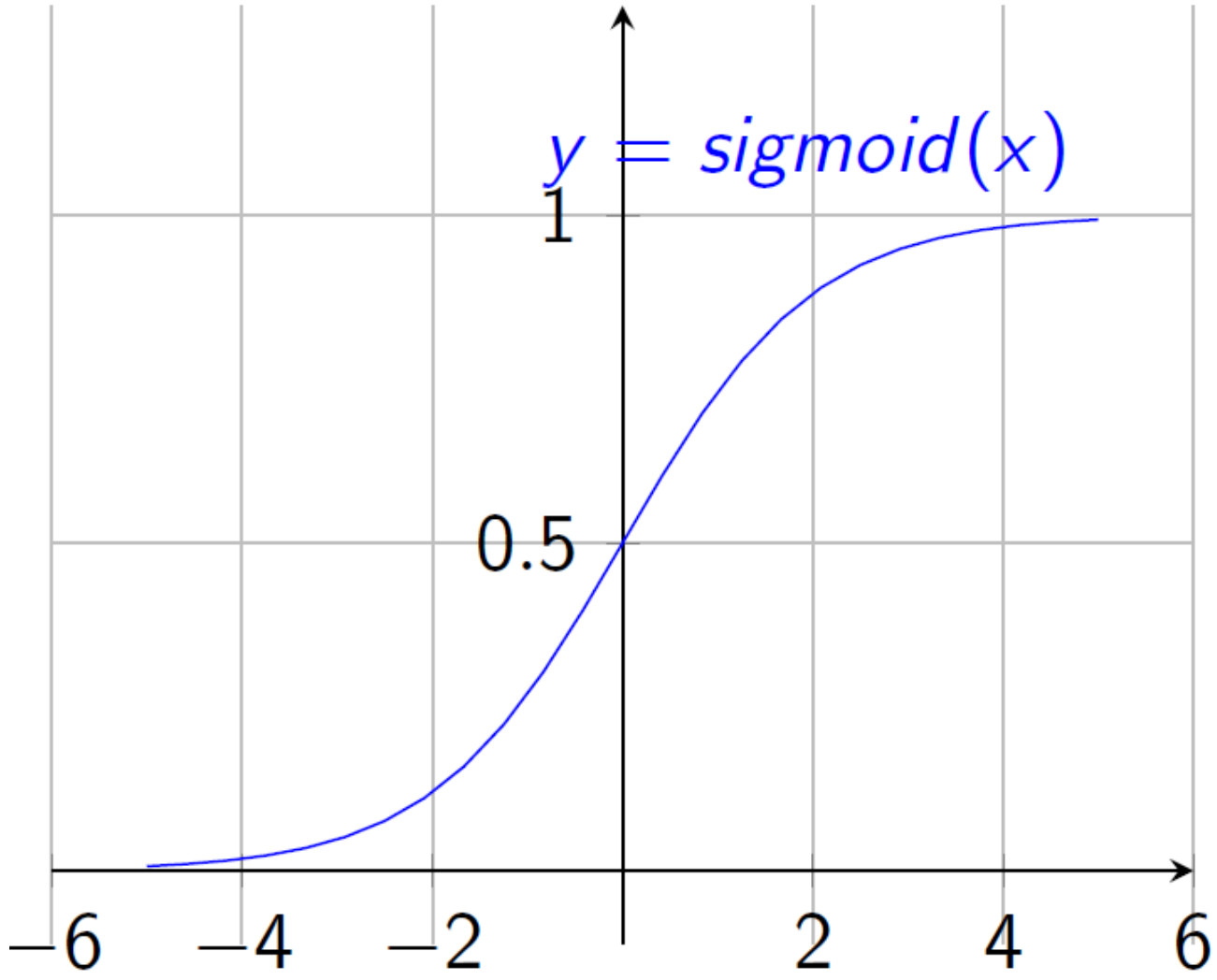
- Output:

$$y = f(a) = \sigma\left(\sum_{i=1}^{n} x_i w_i + b\right)$$



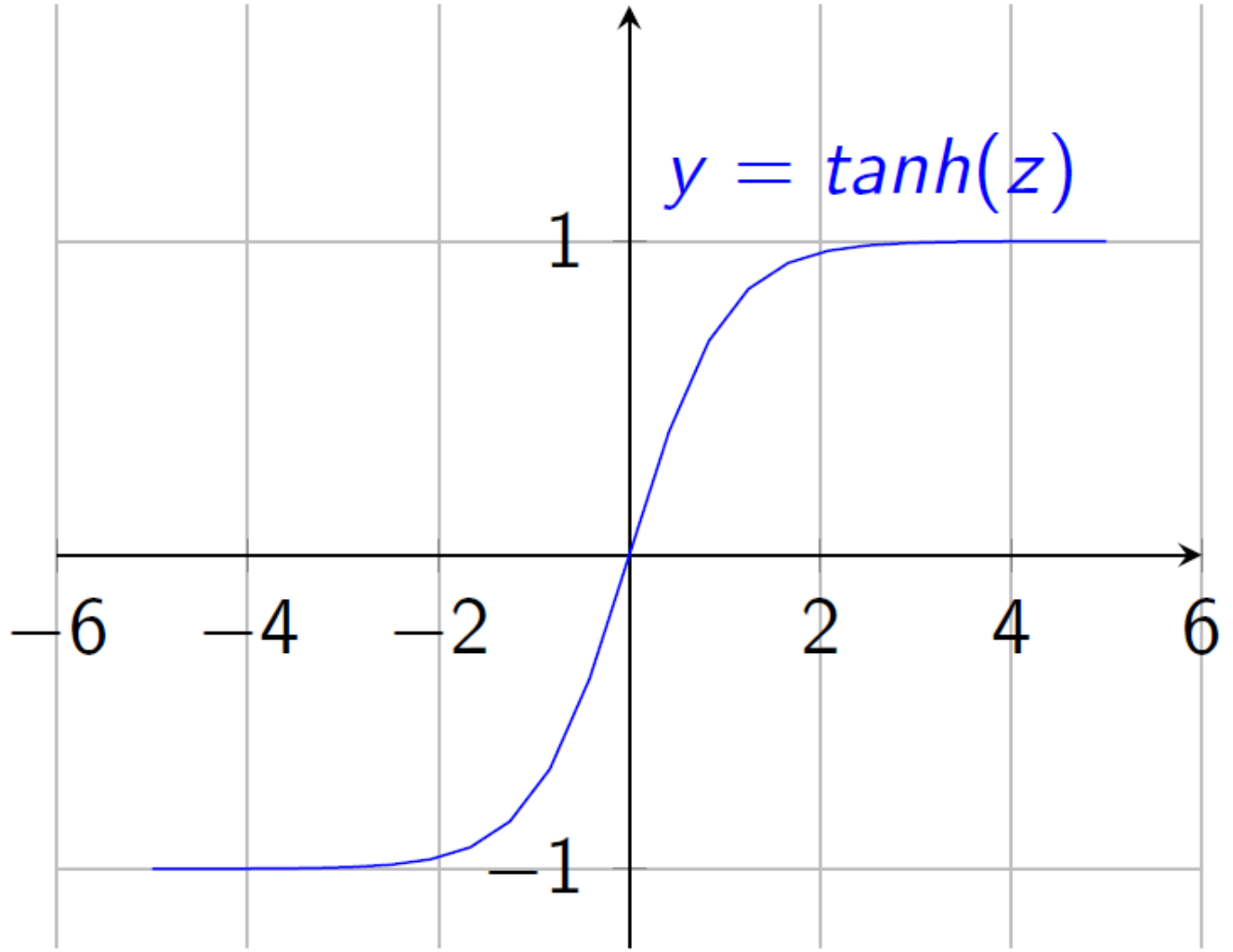
Activation functions: Sigmoid

- $\sigma(z) = \frac{1}{1+\exp -z}$
- Output bounded between [0,1]
- Saturation for large (negative, positive) inputs
  - Gradients go to zero – no information!
- Outputs centered at 0.5
- Not used nowadays



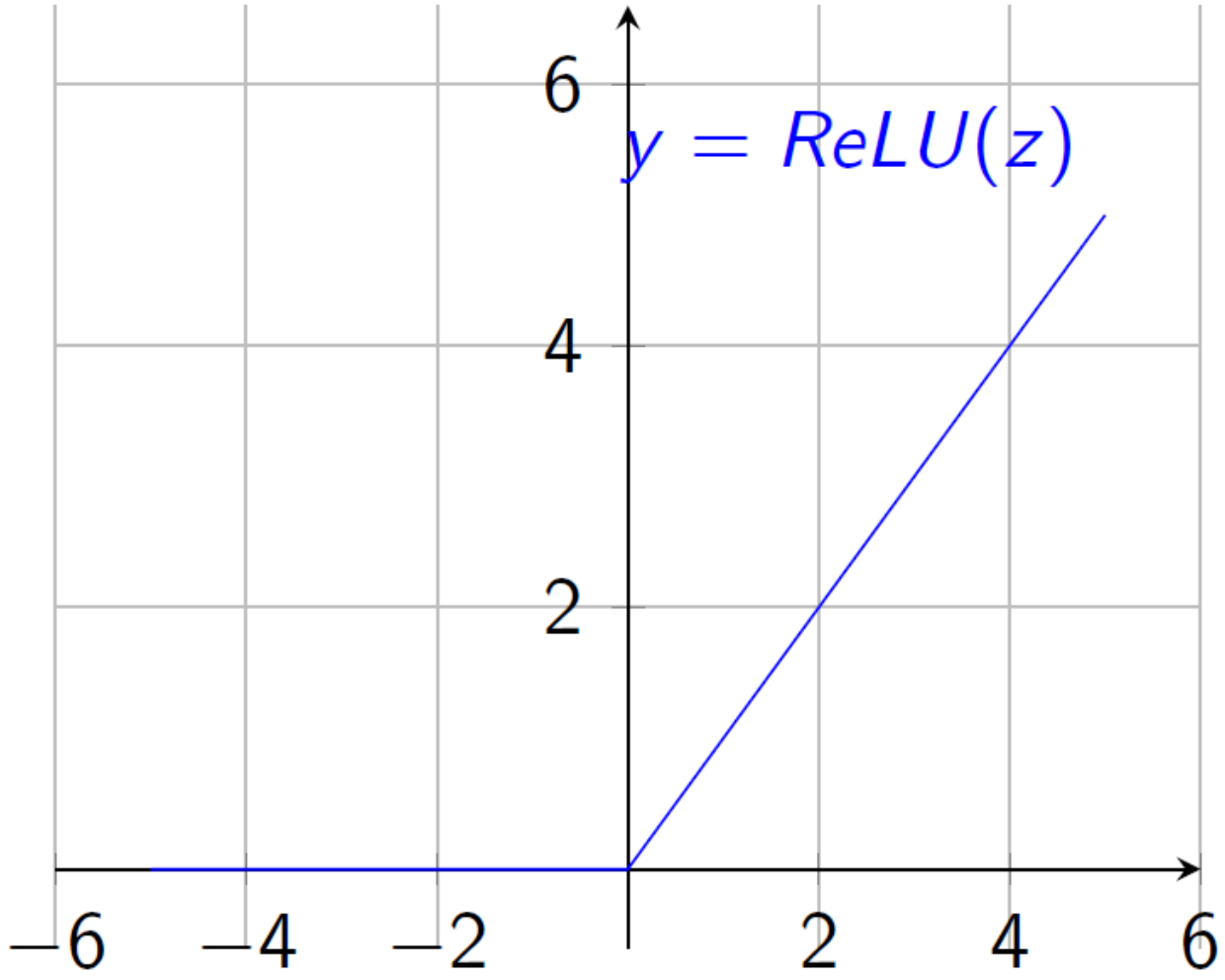
Activation functions: Tanh

- $\sigma(z) = \tanh z$
- Output bounded between [-1,1]
- Saturation for large (negative, positive) inputs
  - Gradients go to zero – no information!
- Outputs centered at 0
- Preferable to sigmoid



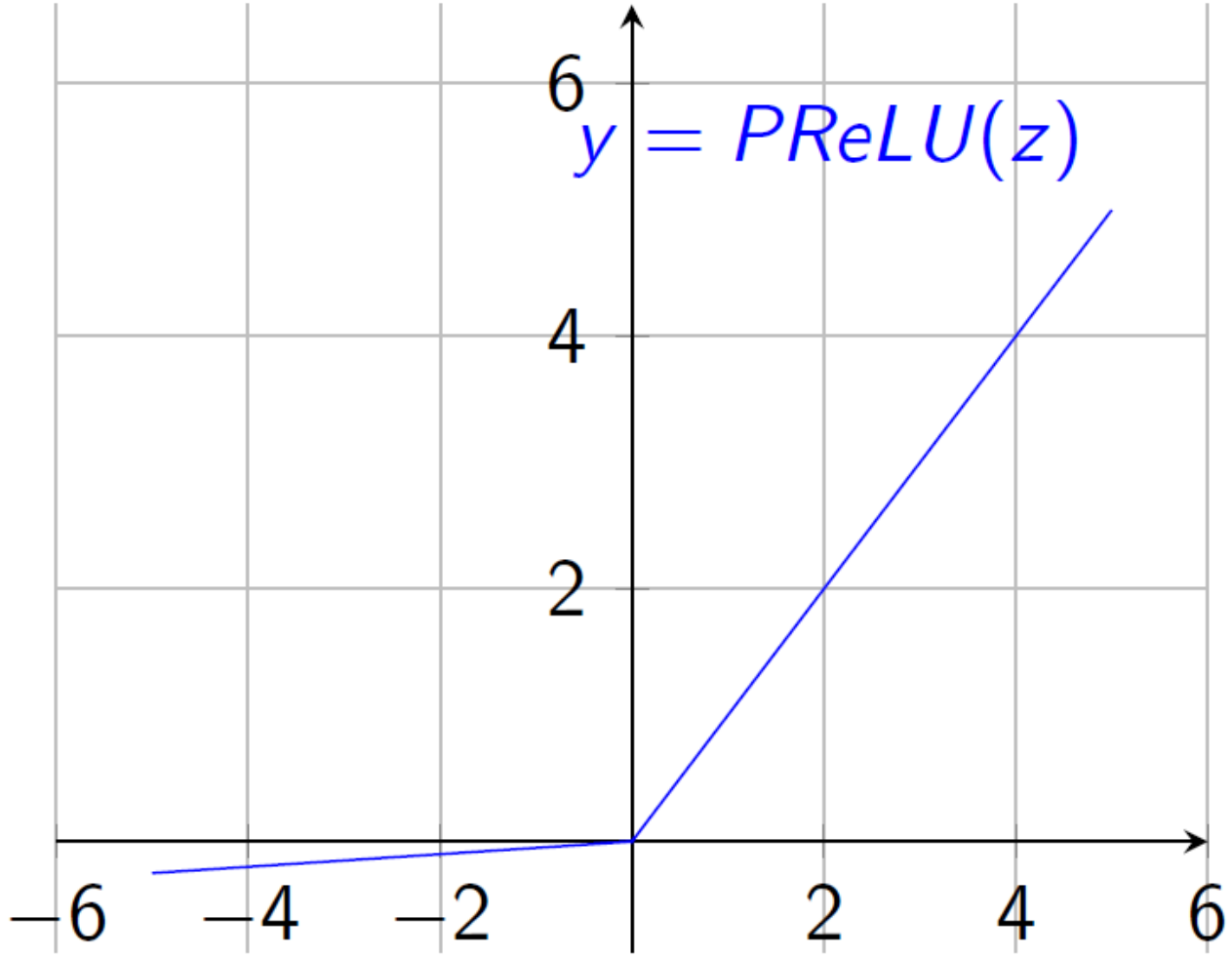
Activation functions: ReLU

- $\sigma(z) = \max(z, 0)$
- Output bounded to [0, +inf]
- Efficient gradients
- Helps on convergence vs tanh
- Drawback: no gradient in negative region, dead forever
- Default choice



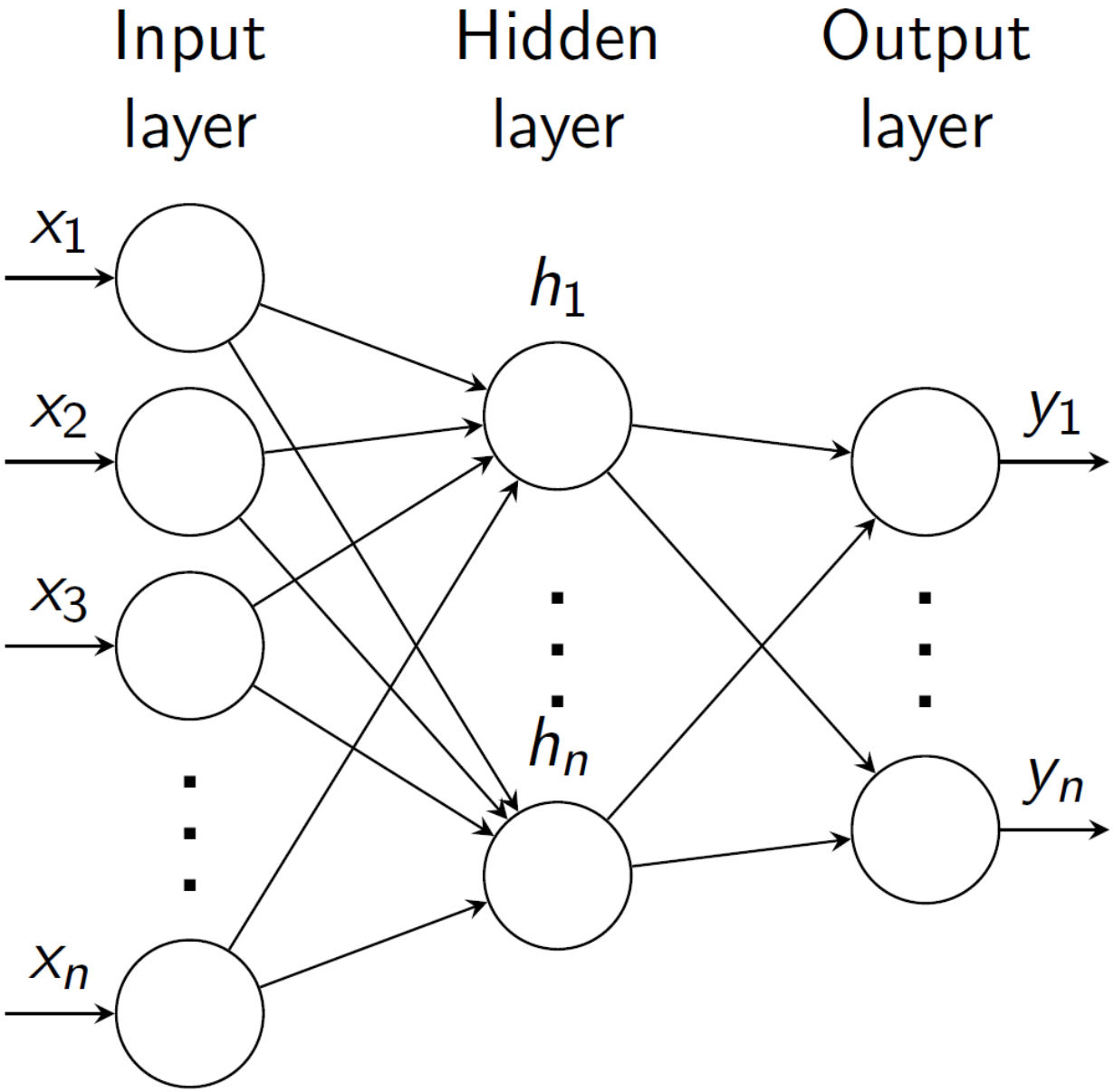
Activation functions: Leaky ReLU

- $\sigma(z) = \begin{cases} z > 0 : z \\ z < 0 : \alpha z \end{cases}$
- α small, e.g. 0.02
- Output bounded to [-inf, +inf]
- Nickname: Probabilistic ReLU (PReLU)
- Non-zero gradients everywhere
- α can be learned during training



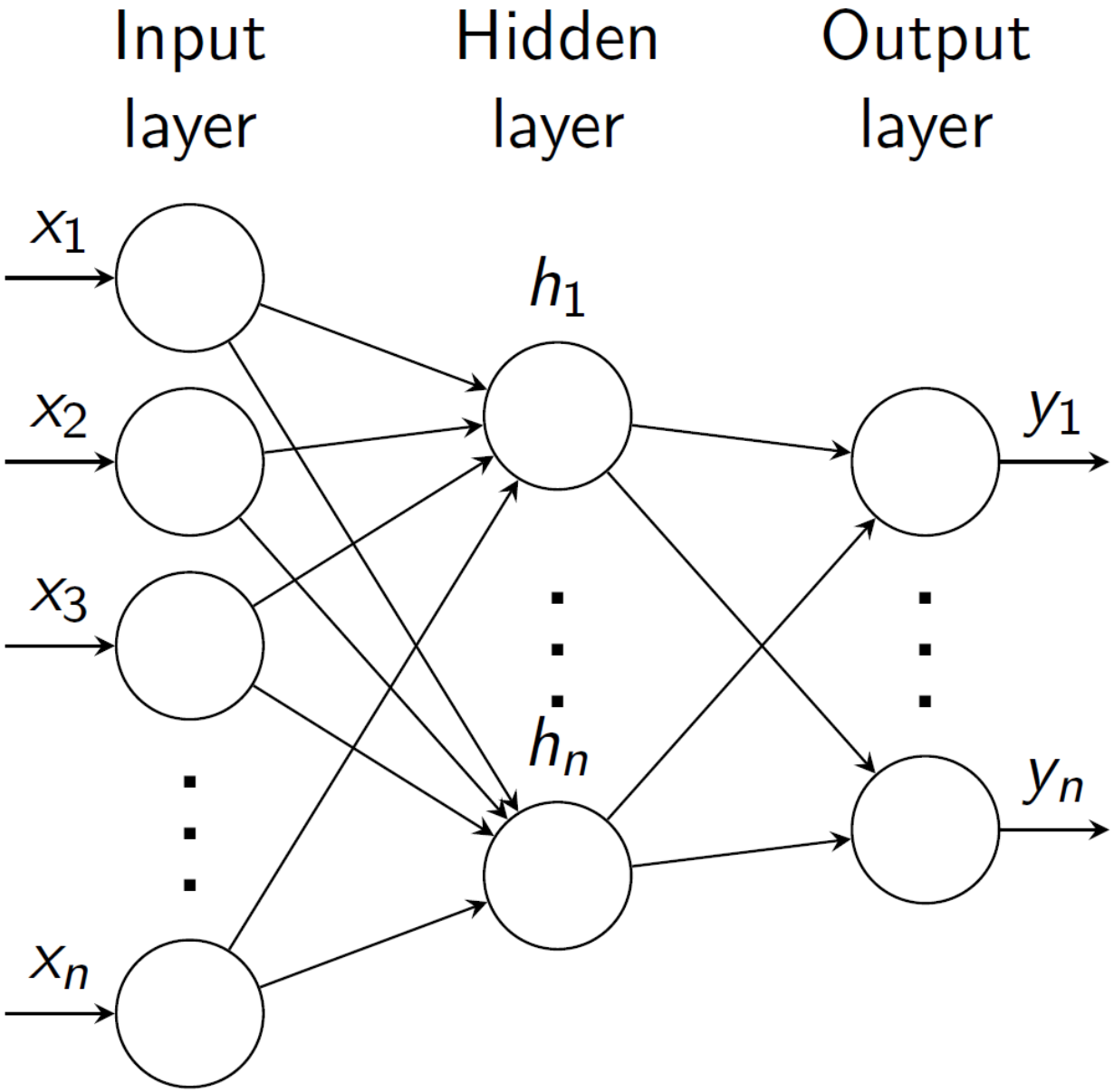
Multiple layers

- Neural networks: Multiple layer of neurons
- Acyclic structure: Full connections between layers
- Layers between input and output: hidden layers
- Also known as:
  - Artificial Neural Networks (ANN)
  - Multi-layer Perceptron
  - Fully-connected network



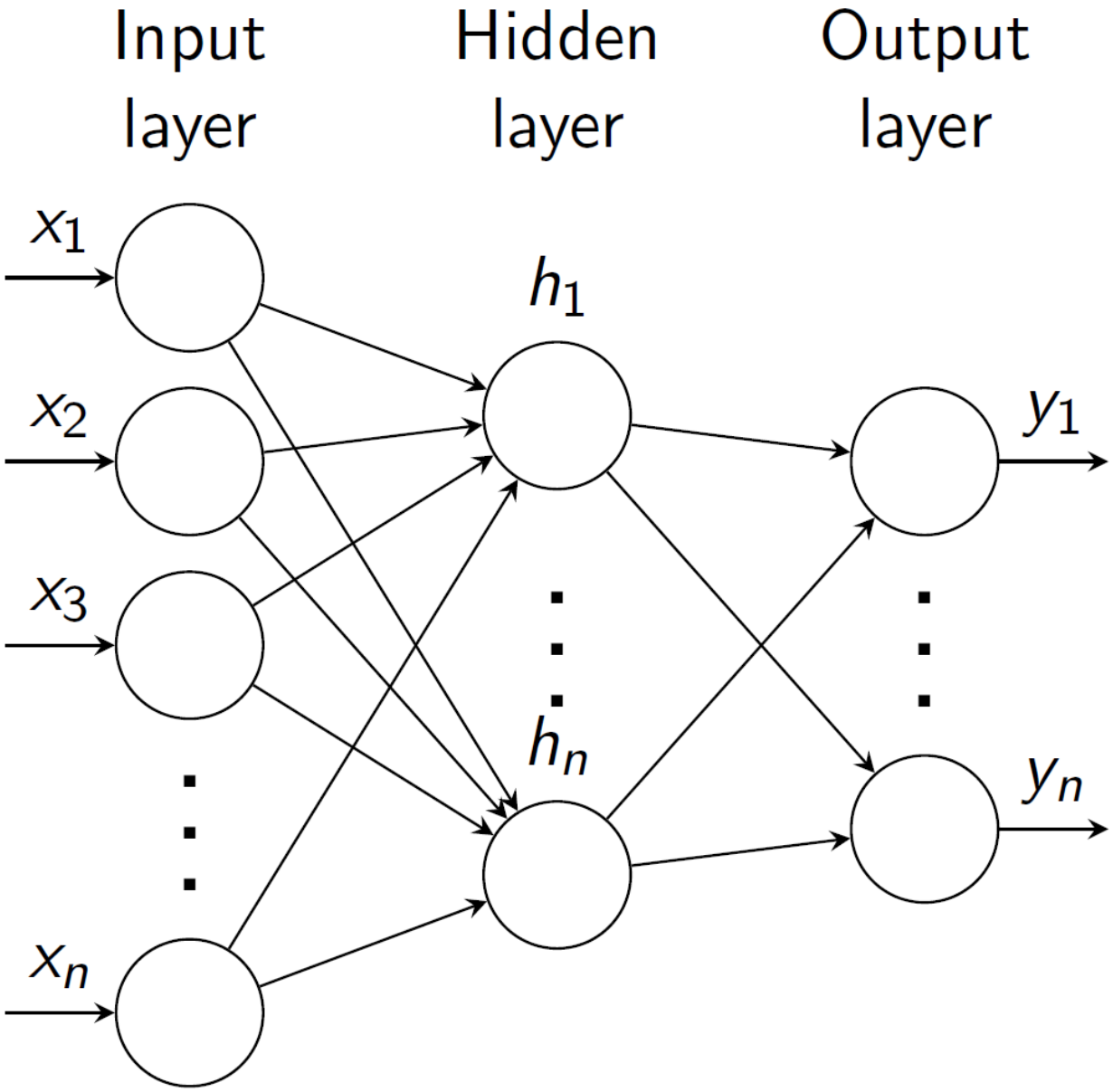
Multiple layers

- How many layers in this example?
- By convention:
  - Number of layers = hidden + output
- Parameters:
  - Weight matrices: W^1, W^2
  - Bias vectors: b^1, b^2



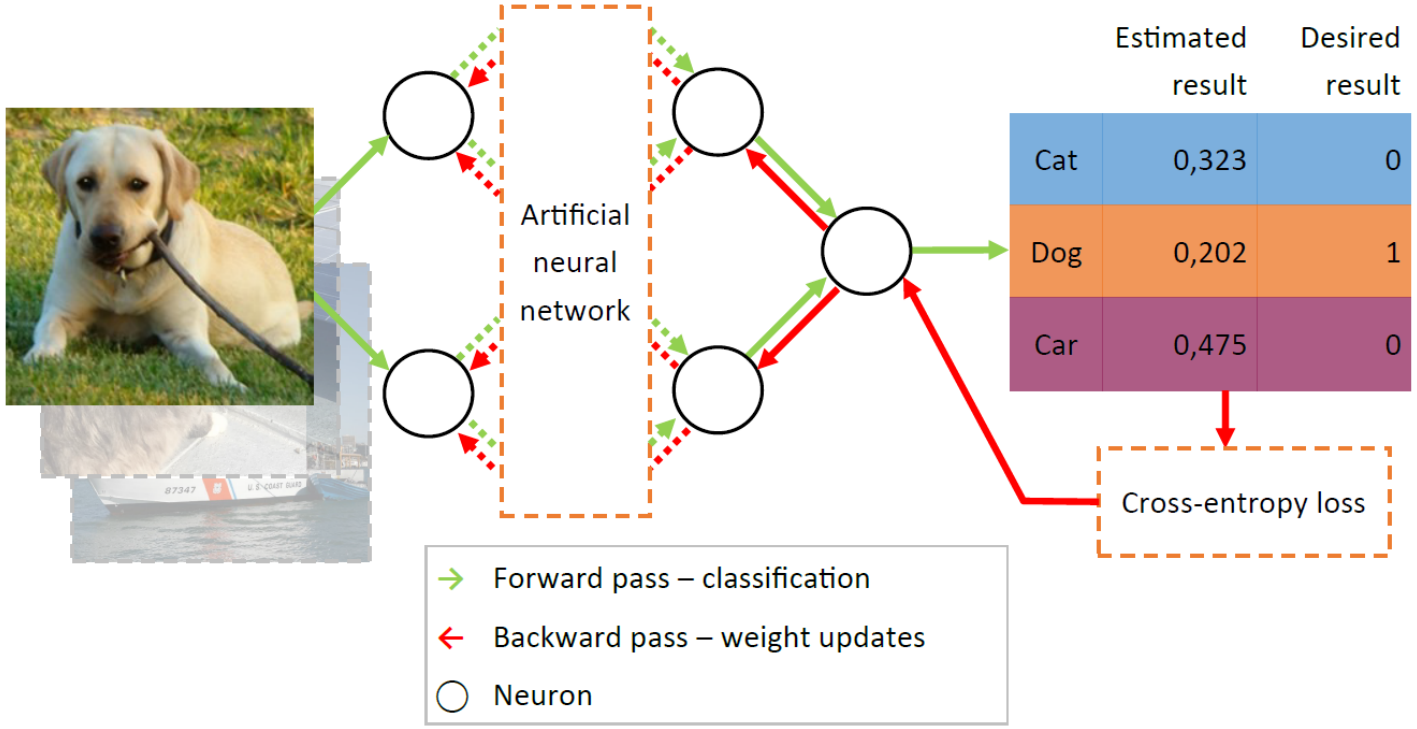
Multiple layers

- In theory, more than two layers can represent any function
  - Infeasible in current practice, though ☺



Optimization algorithms

- Network is initialized with random weights
- Input is supervised, i.e. with manually annotated labels
- Define loss function to describe prediction error
- Propagate error through network to update weights and biases



Optimization algorithms

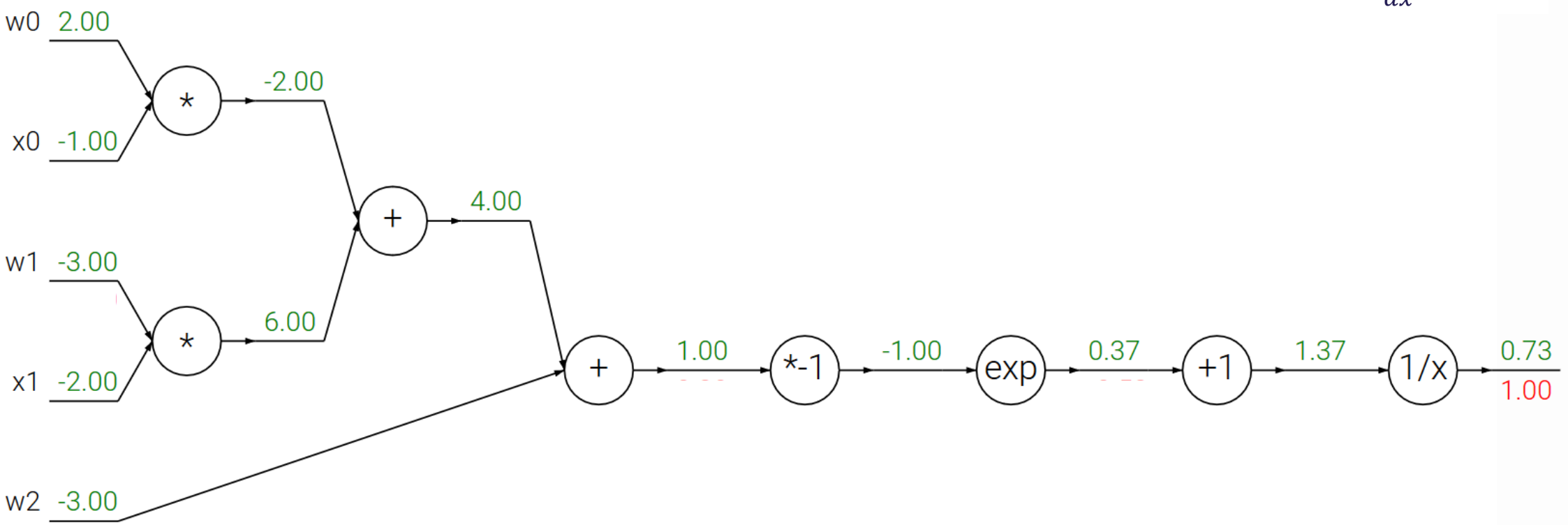
$$f(x) = x^{-1} \rightarrow \frac{df}{dx} =$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} =$$

$$f_c(x) = \exp(x) \rightarrow \frac{df}{dx} =$$

$$f_c(x) = ax \rightarrow \frac{df}{dx} =$$

- Gradients propagate through the network according to the chain rule



Optimization algorithms

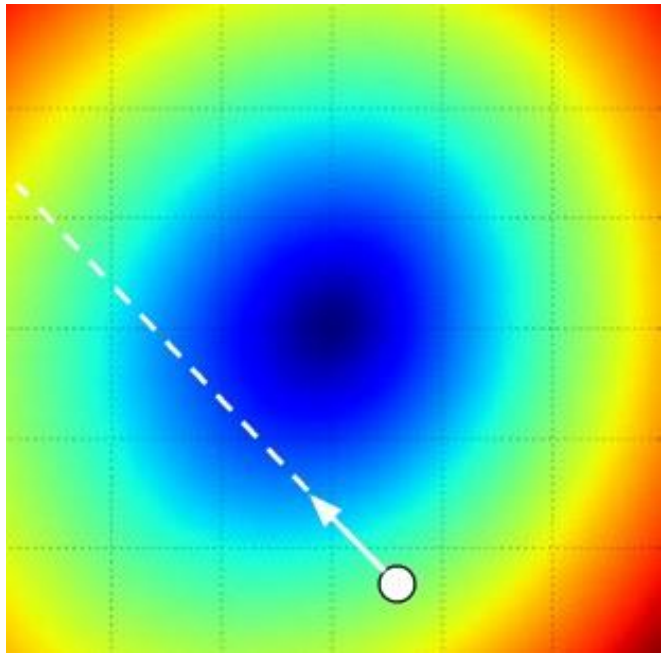
- Gradients propagate through the network according to the chain rule
- Real networks: Not feasible to calculate gradient analytically
- Instead: Stochastic Gradient Descent uses single sample to compute the numerical gradient
- Use several mini-batches of data to calculate stable gradient
- Choose a proper learning rate

$$f(x) = x^{-1} \rightarrow \frac{df}{dx} = -x^{-2}$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

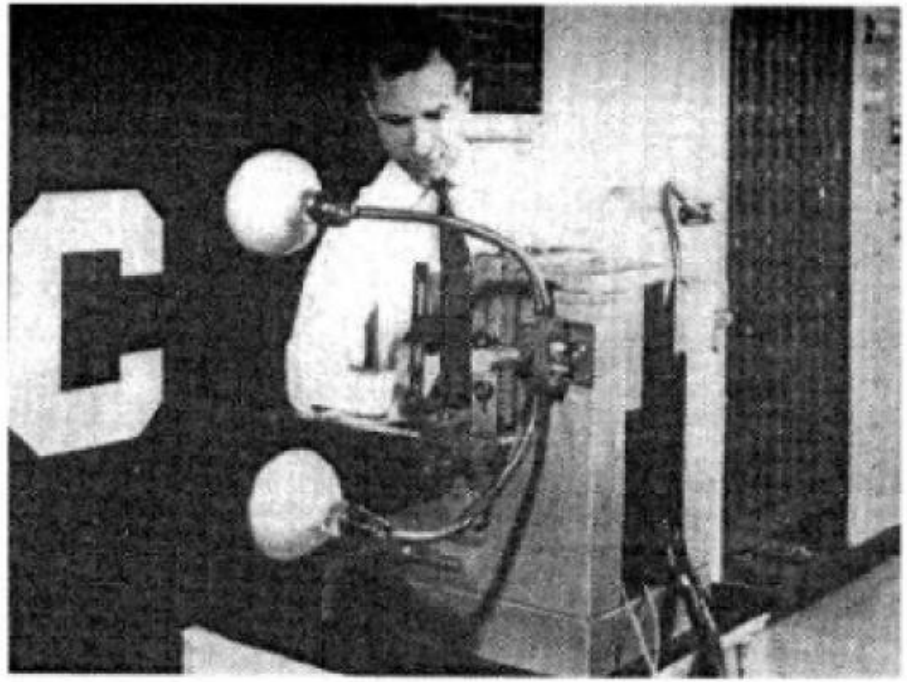
$$f_c(x) = \exp(x) \rightarrow \frac{df}{dx} = \exp(x)$$

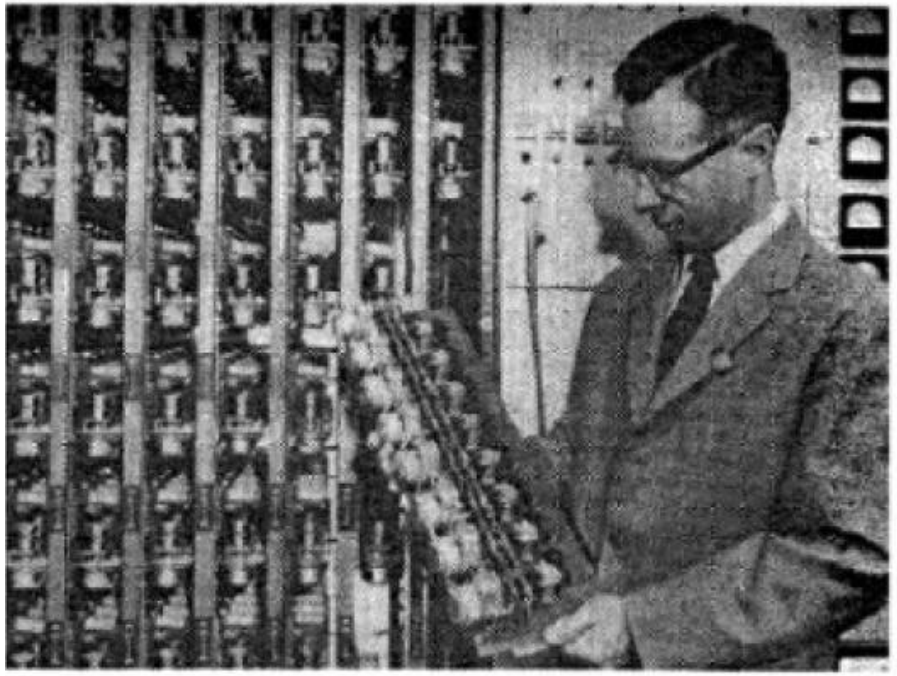
$$f_c(x) = ax \rightarrow \frac{df}{dx} = a$$



Historical Neural Networks

- 1950s: Perceptrons [Rosenblatt]
- 1986: Back-propagation [Rumelhart, Hinton, Williams]
- 1989: Convolutional Networks [LeCun et al.]
- 1997: Recurrent Neural Networks (LSTM) [Hochreiter and Schmidhuber]
- 2011: Neural Networks and Speech [Hinton et al.]
- 2012: Convolutional Neural Networks applied to ImageNet (AlexNet) [Krizhevsky, Sutskever, Hinton]





Convolutional Neural Networks



First Convolutional Neural Networks

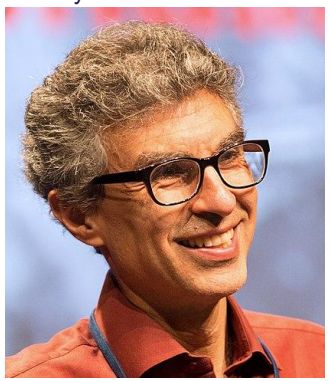
- Yann LeCun et al., 1998
  - Neural network with specialized connectivity structure
  - Higher stages compute more global, more invariant features

Yann LeCun

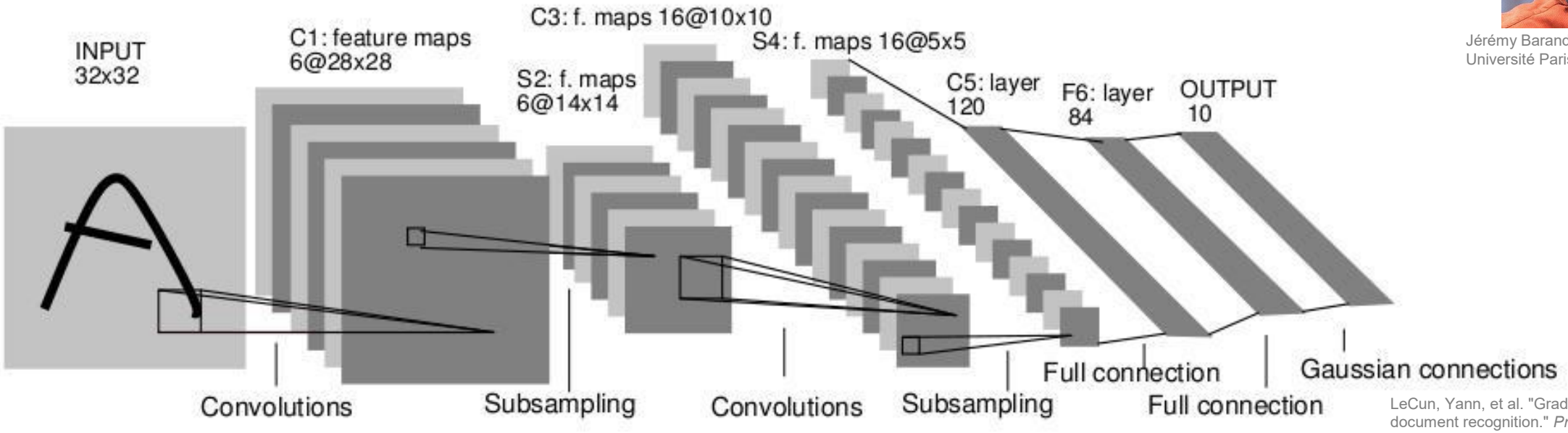


https://www.linkedin.com/feed/update/urn:li:activity:7504209134263902209/

Yoshua Bengio



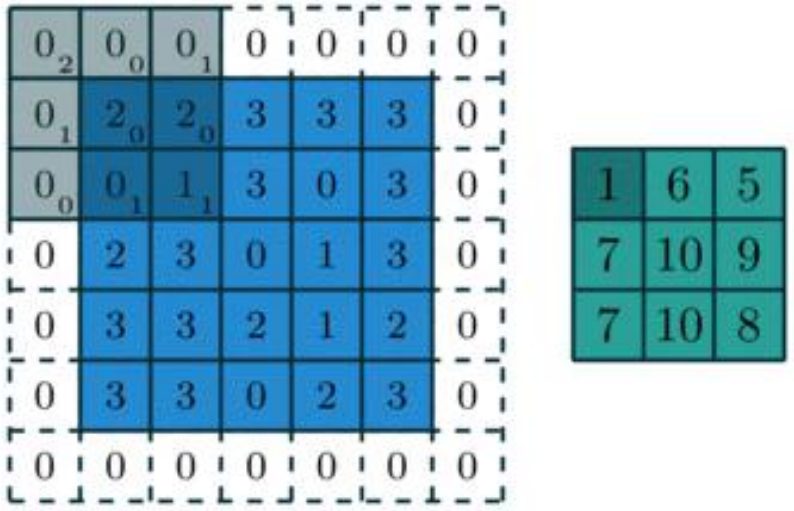
Jérémy Barande / Ecole polytechnique Université Paris-Saclay / CC BY-SA 2.0



LeCun, Yann, et al. "Gradient-based learning applied to document recognition." *Proceedings of the IEEE* 86.11 (1998): 2278-2324.

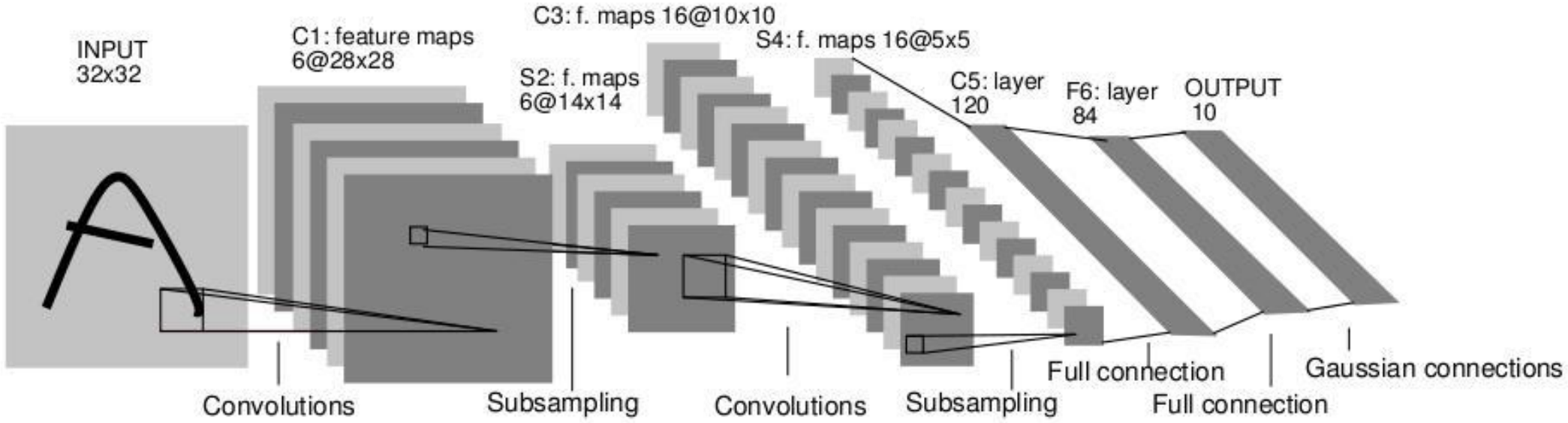
ConvNet architecture

- Exploits two properties of images:
  - Dependencies are local
    - No need to have each unit connect to every pixel
  - Spatially stationary statistics
    - Translation invariant dependencies
    - Only approximately true



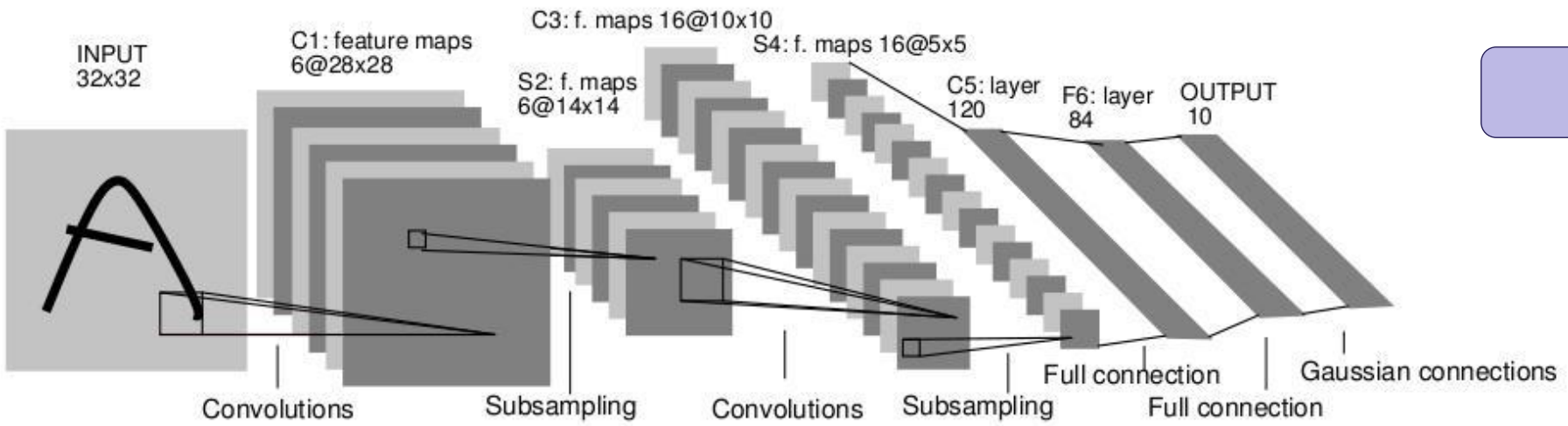
ConvNet architecture

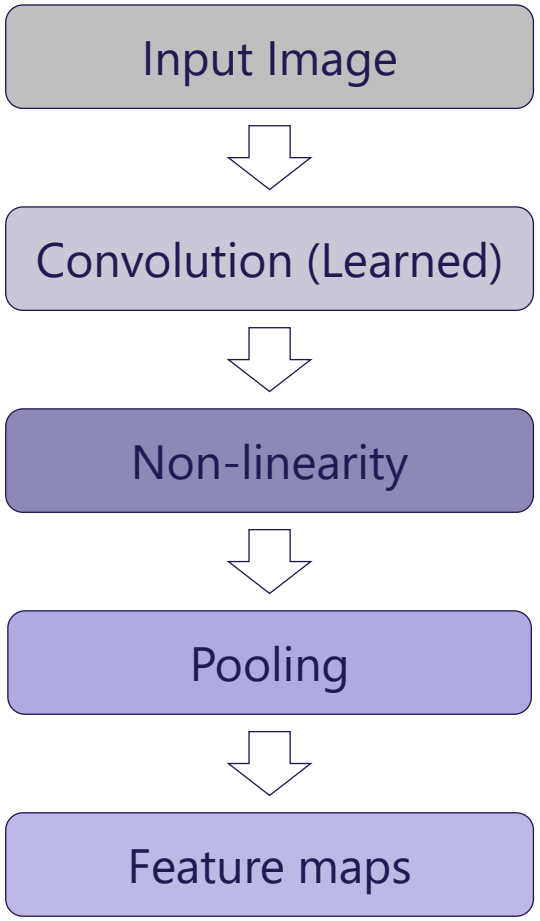
- Stack multiple stages of simple cells
- Higher stages compute more global, more invariant features
- Classification layer on top



ConvNet architecture

- Feed-forward
- Fully supervised
- Train convolutional filters by back-propagating classification errors



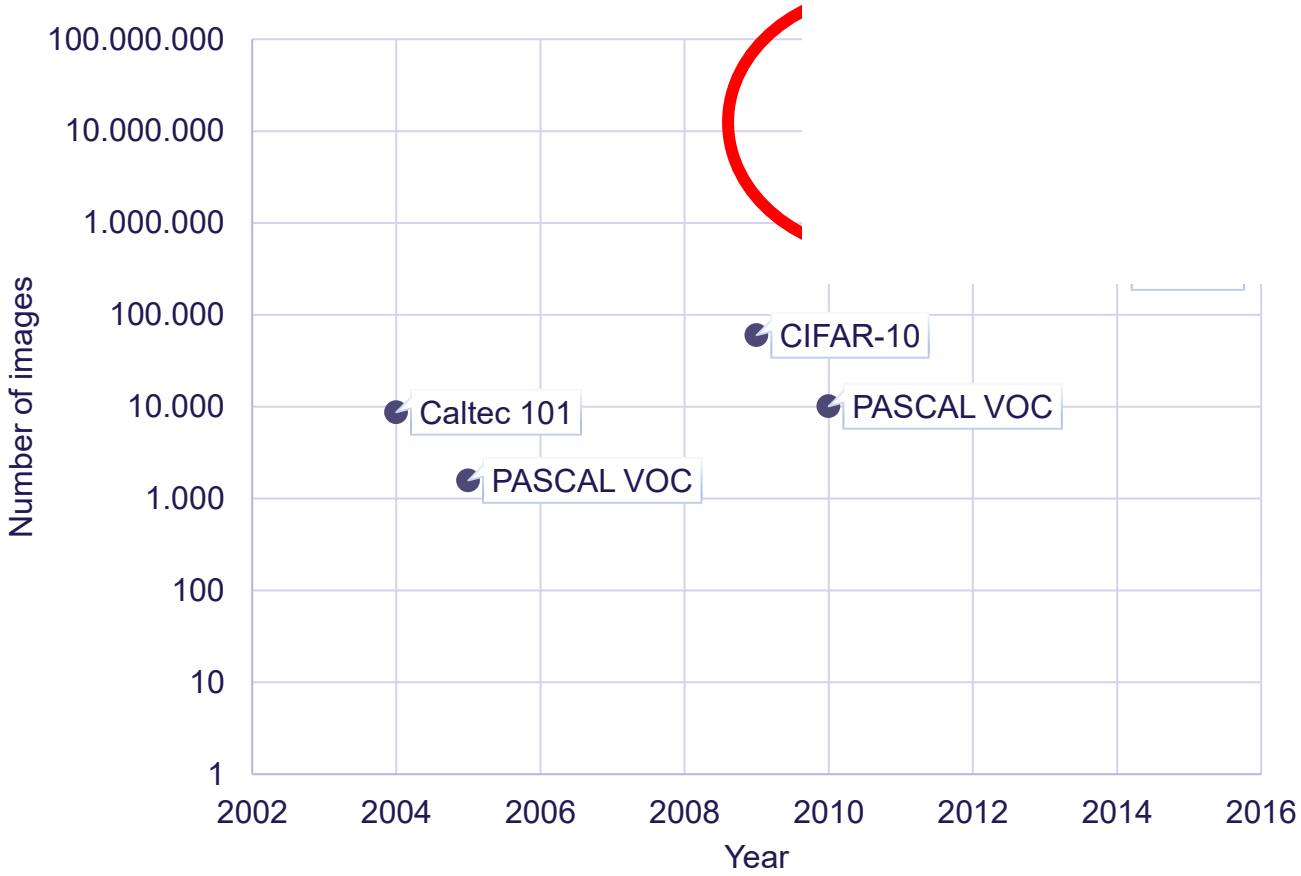


Datasets

- ImageNet
  - 14 million images
  - 20 000 classes
  - Images from Internet (Flickr)
  - Human labels from Amazon Mechanical Turk



Fei-Fei Li



Deng, Jia, et al. "Imagenet: A large-scale hierarchical image database." *2009 IEEE conference on computer vision and pattern recognition*. IEEE, 2009.

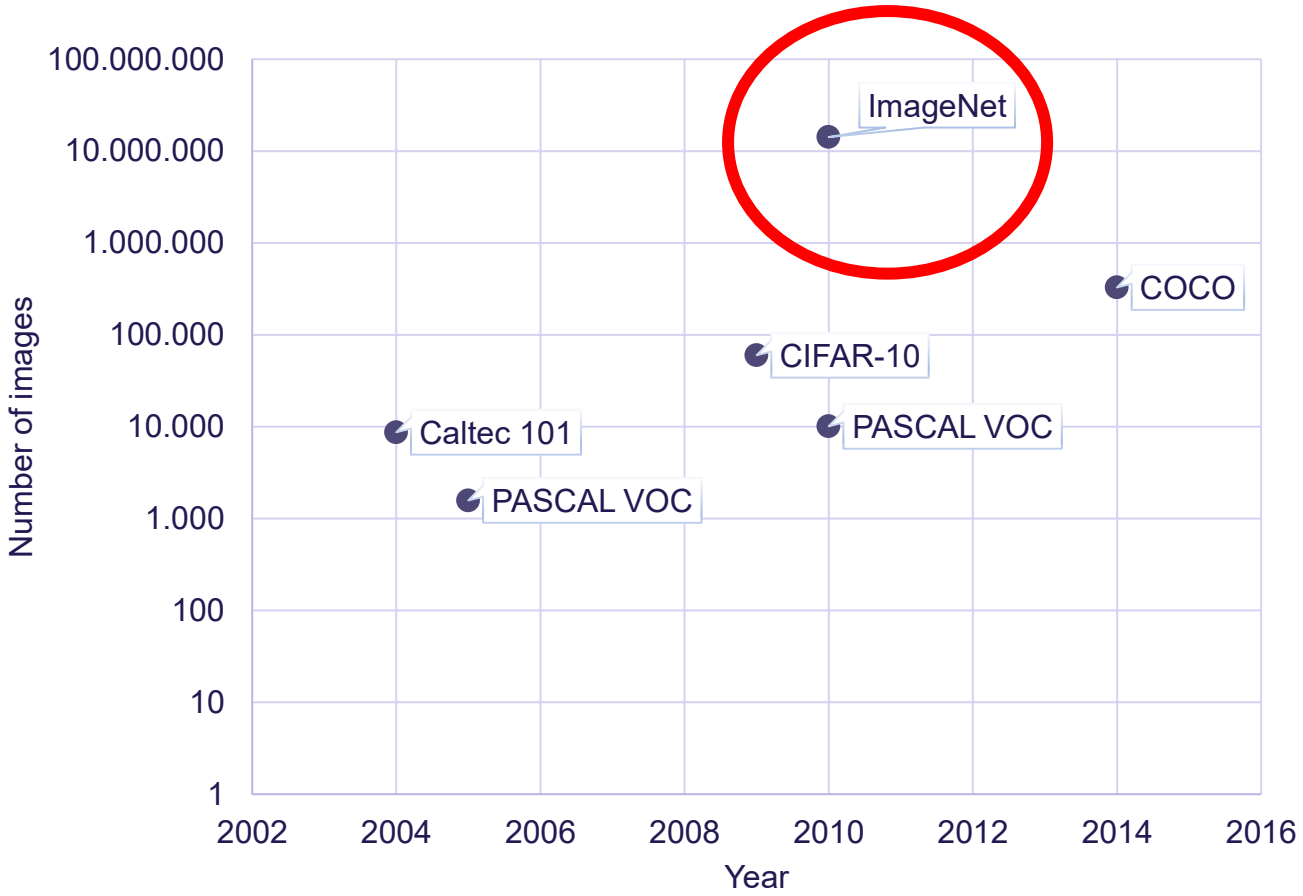
ImageNet

- Pixels → Class label

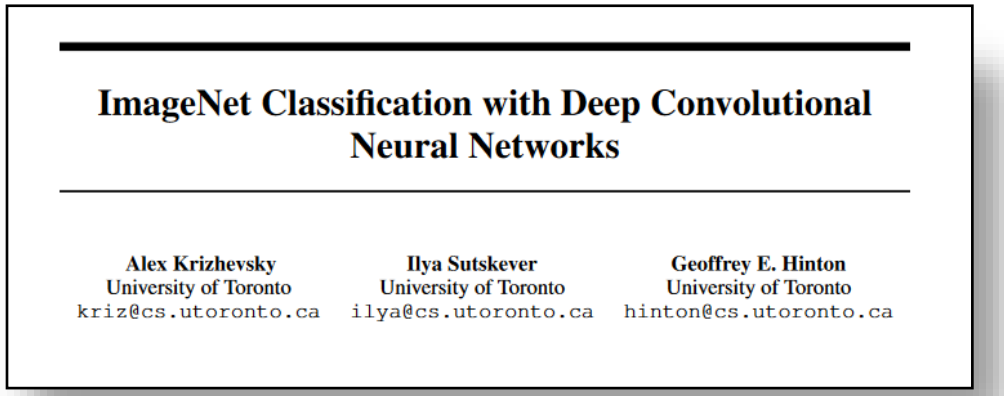


ImageNet

- 14 million images
- 20 000 classes
- Images from Internet (Flickr)
- Human labels from Amazon Mechanical Turk



AlexNet



ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

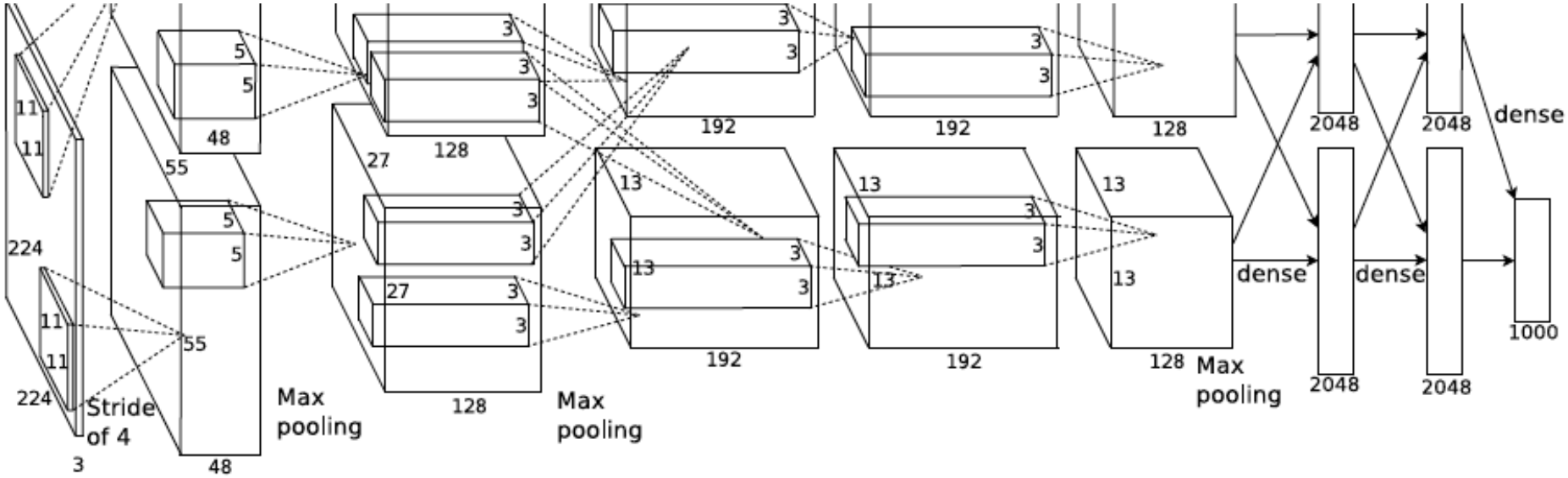
Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

AlexNet

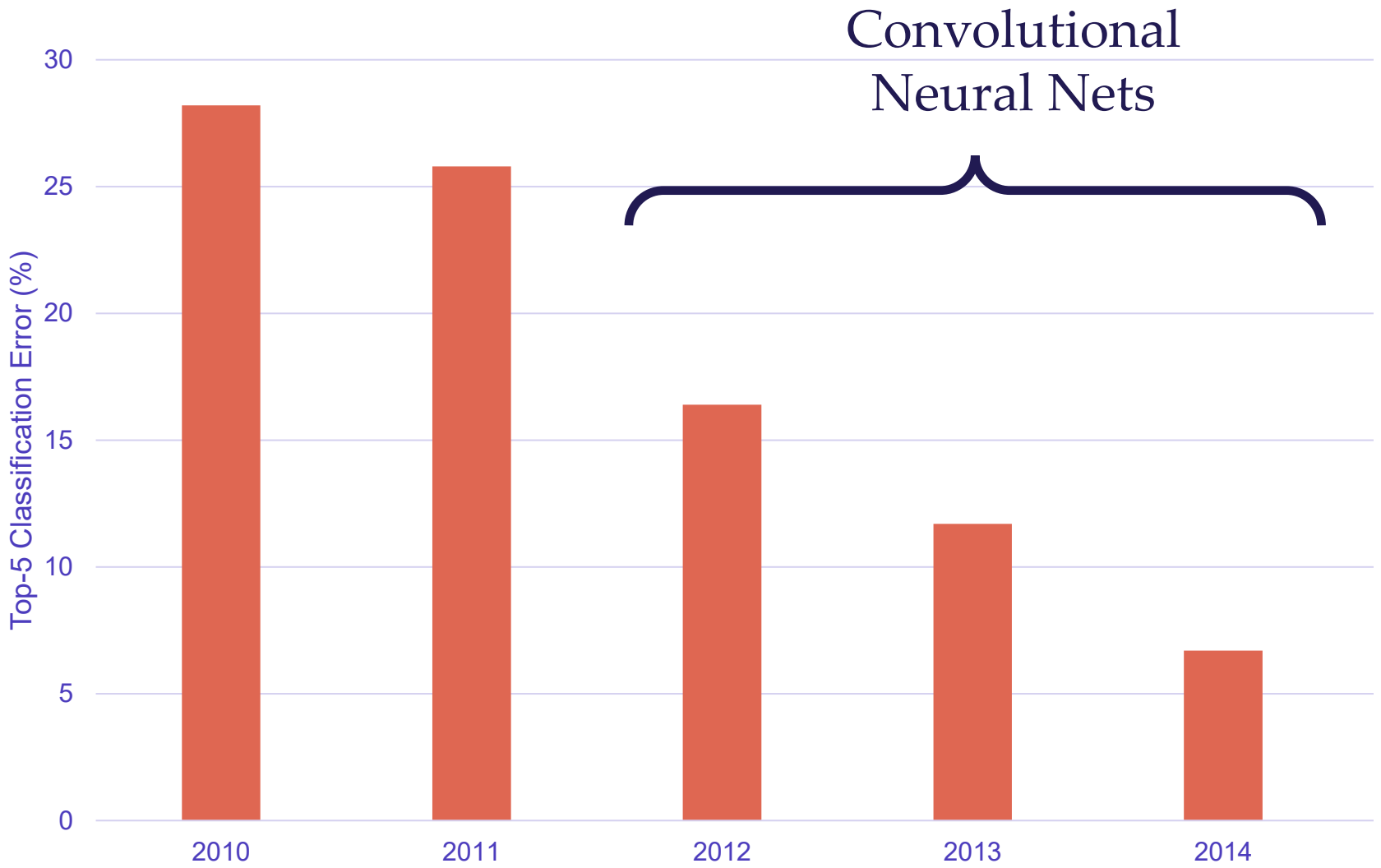
- Same model as LeCun'98 but:
  - Bigger model (8 layers)
  - More data (ImageNet)
  - GPU implementation
    (50x speedup over CPU)

- Better regularization (Dropout)
- 7 hidden layers, 650,000 neurons, 60,000,000 parameters
- Trained on 2 GPUs for a week

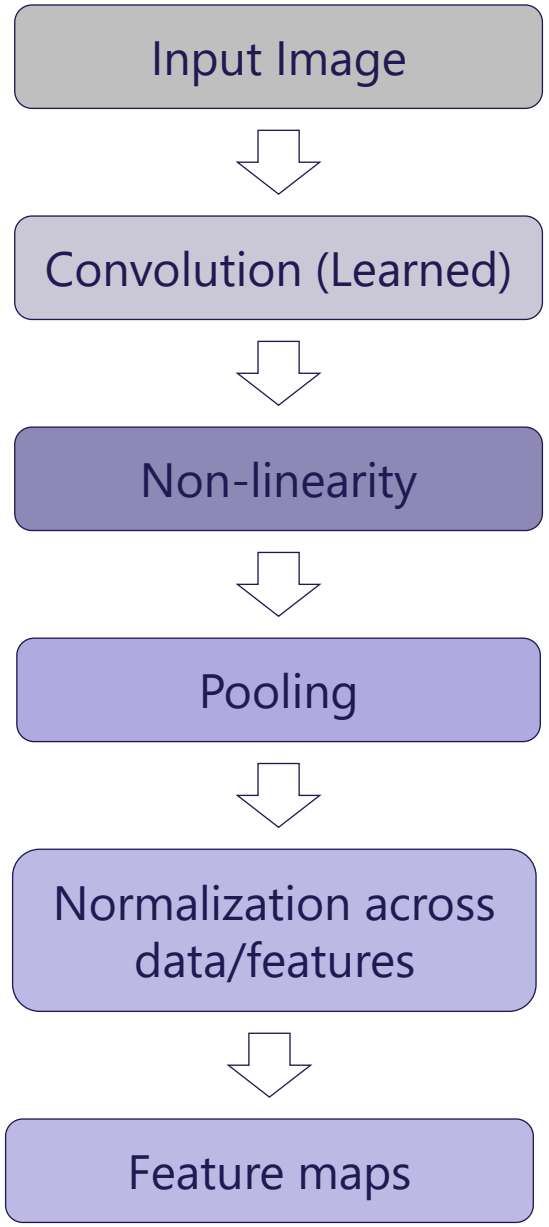


Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." *Advances in neural information processing systems*. 2012.

ImageNet Classification (2010 – 2014)

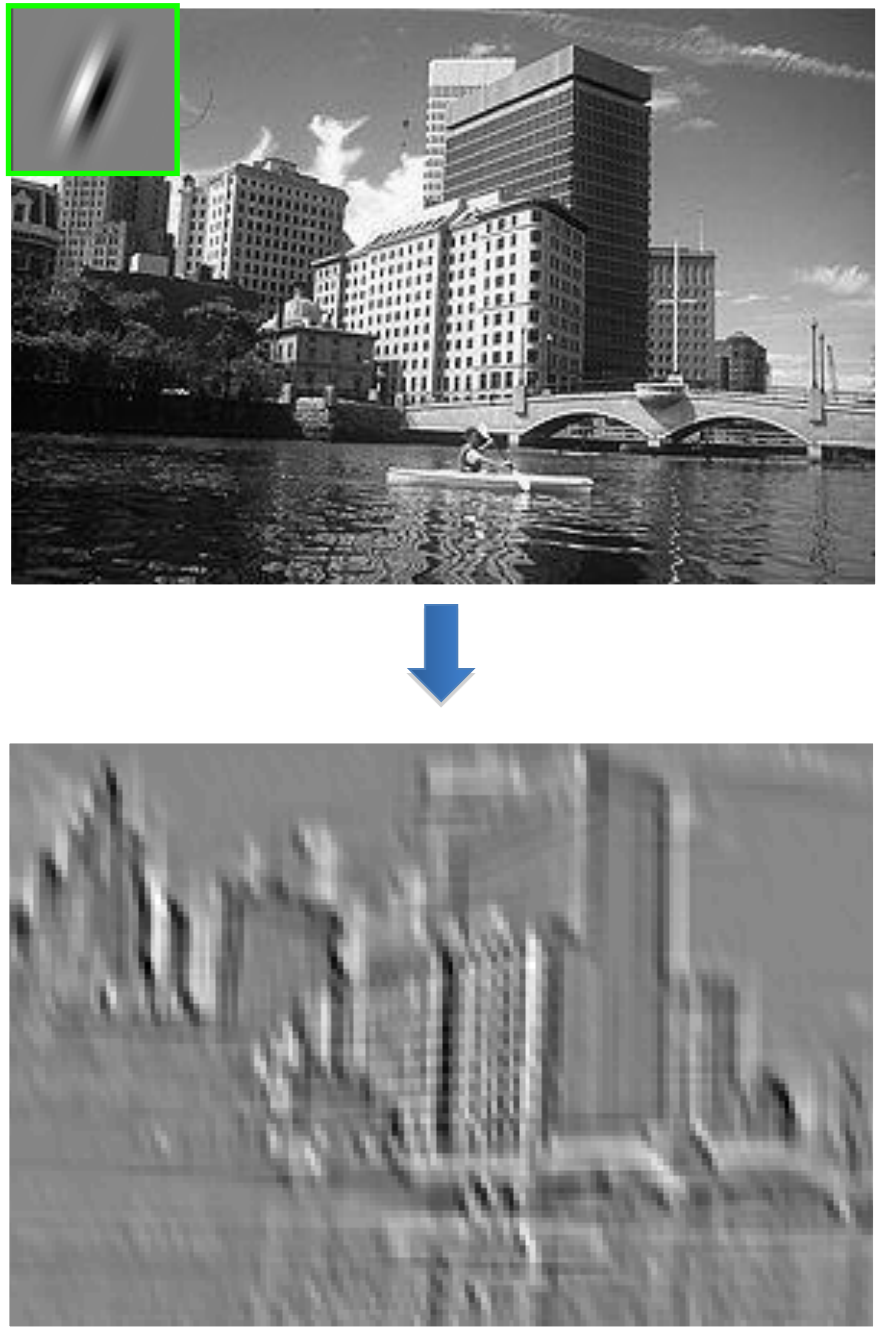


Component walkthrough



Filtering / Convolution

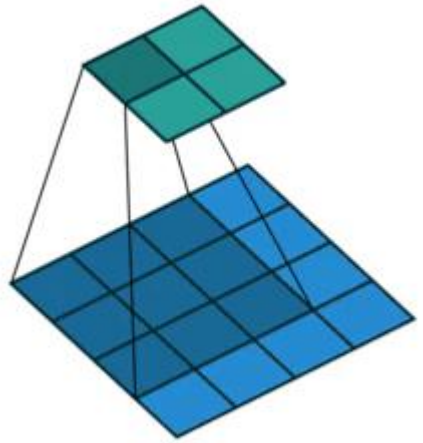
- Linear combination of input and filter
- Same filter at each location
- # Parameters: F x F x D x K + K
  - F filter size
  - D input depth
  - K number of filters



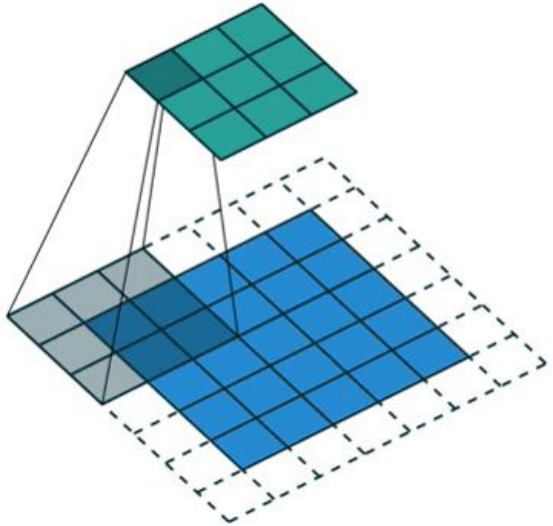
Filtering / Convolution

- Linear combination of input and filter
- Same filter at each location
- # Parameters: F x F x D x K + K
  - F filter size
  - D input depth
  - K number of filters
- Output size:
  - Filter size
  - Padding
  - Stride

Padding = 0
Stride = 1



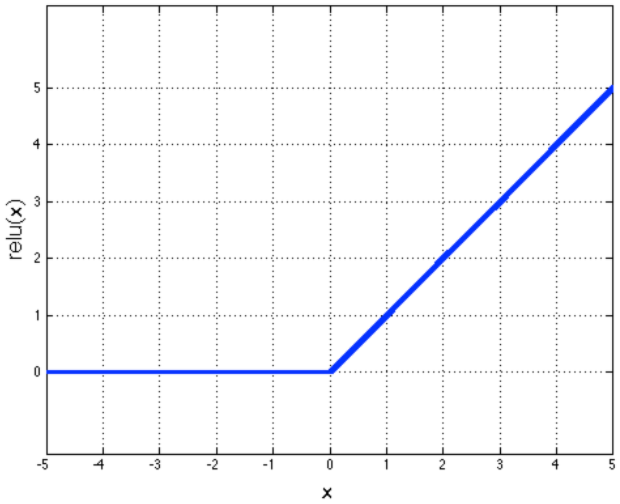
Padding = 1
stride = 2



Non-linear activation functions

- Applied per-pixel
- ReLU most popular – faster training, more efficient

- ReLU: f(x) = max(0, x)

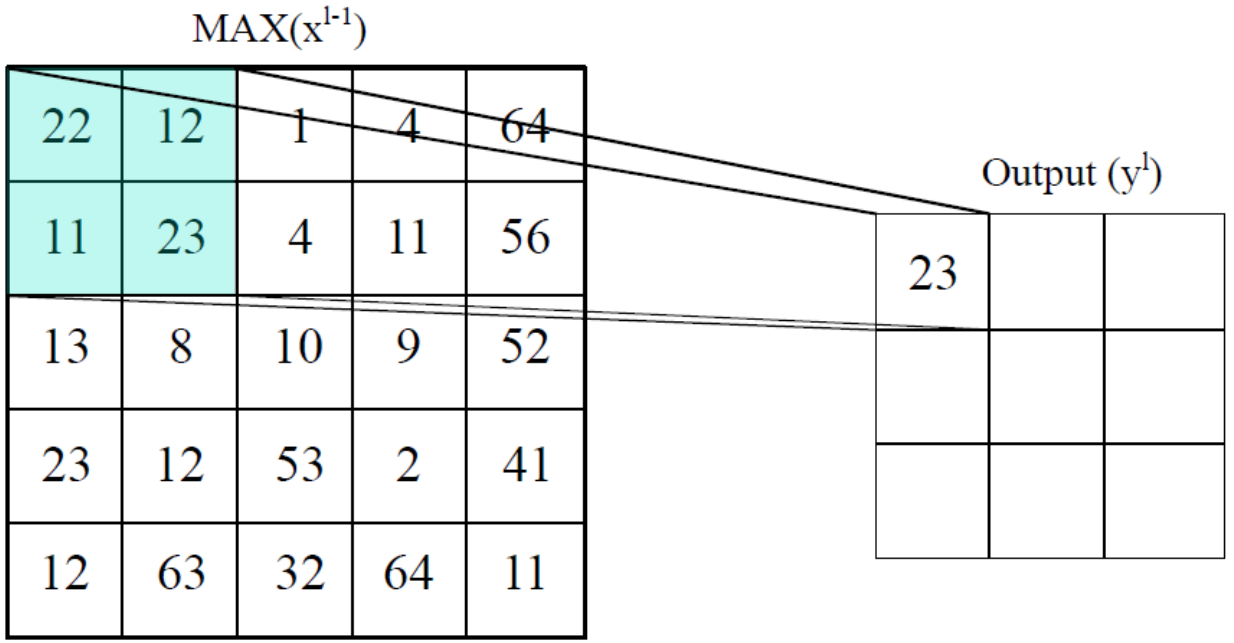






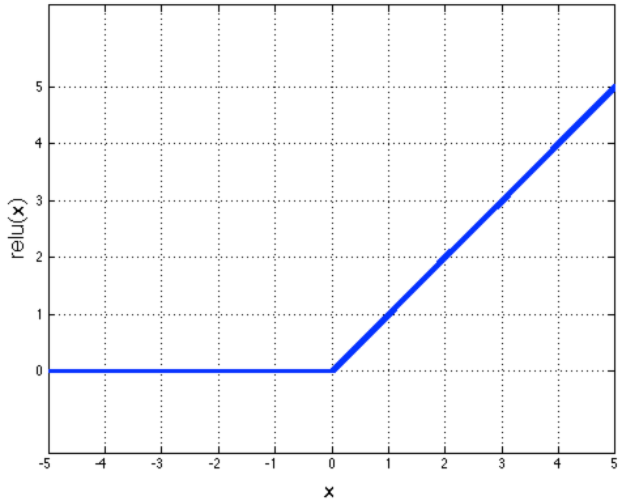
Pooling

- Spatial pooling
  - Non-overlapping or overlapping regions
  - **MAX-pooling**
  - Average pooling
  - Sum-pooling
- Hyper-parameter – not learnable



Batch normalization

- Data-driven normalization
  - For each layer
  - For each mini-batch
  - Introduced in 2015
- Virtues:
  - Greatly accelerate training
  - Less sensitive to initialization
  - Improve regularization



Batch normalization

$$x \rightarrow \hat{x} = \frac{x - \mu}{\sigma} \rightarrow y = \gamma\hat{x} + \beta$$

- μ: mean of x in mini-batch
- σ: std of x in mini-batch
- γ: scale
- β: shift

- μ, σ: functions of x, analogous to responses
- γ, β: parameters to be learned, analogous to weights

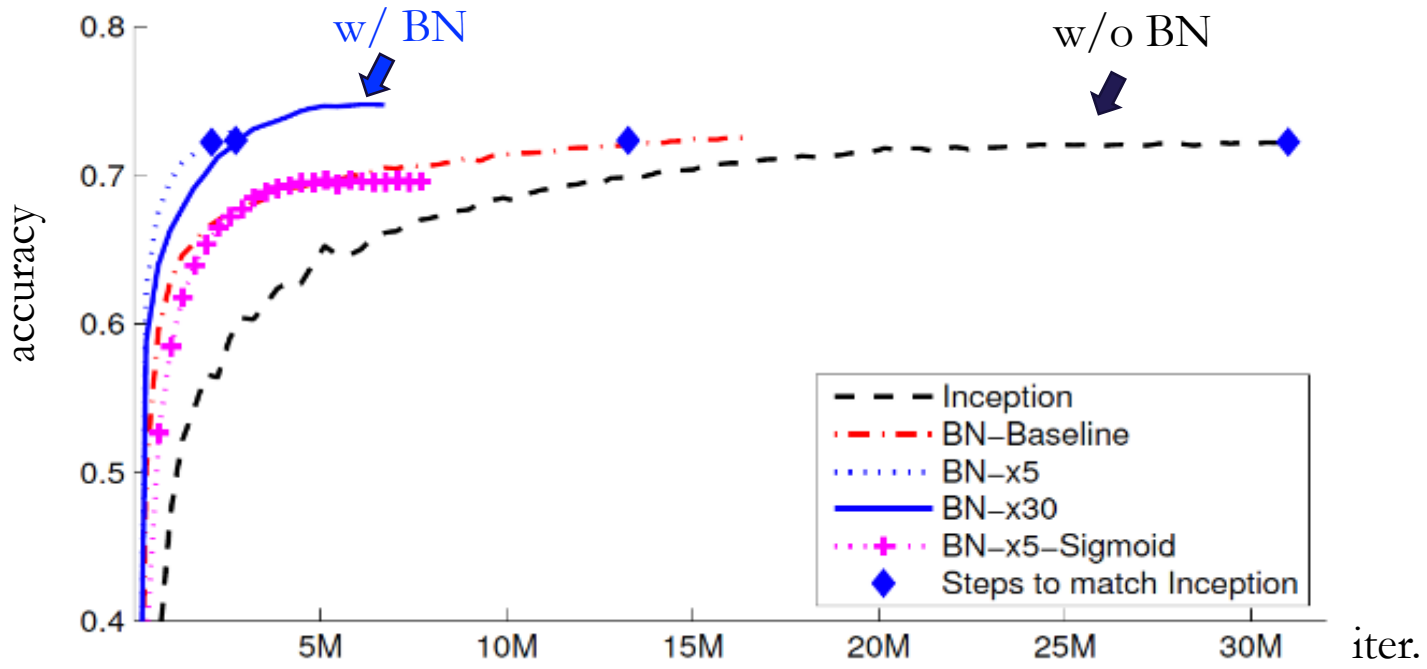


Batch normalization

$$x \rightarrow \hat{x} = \frac{x - \mu}{\sigma} \rightarrow y = \gamma\hat{x} + \beta$$

- 2 modes of BN:
  - Train mode:
    μ, σ are functions of a batch of x
  - Test mode:
    μ, σ are pre-computed on training set

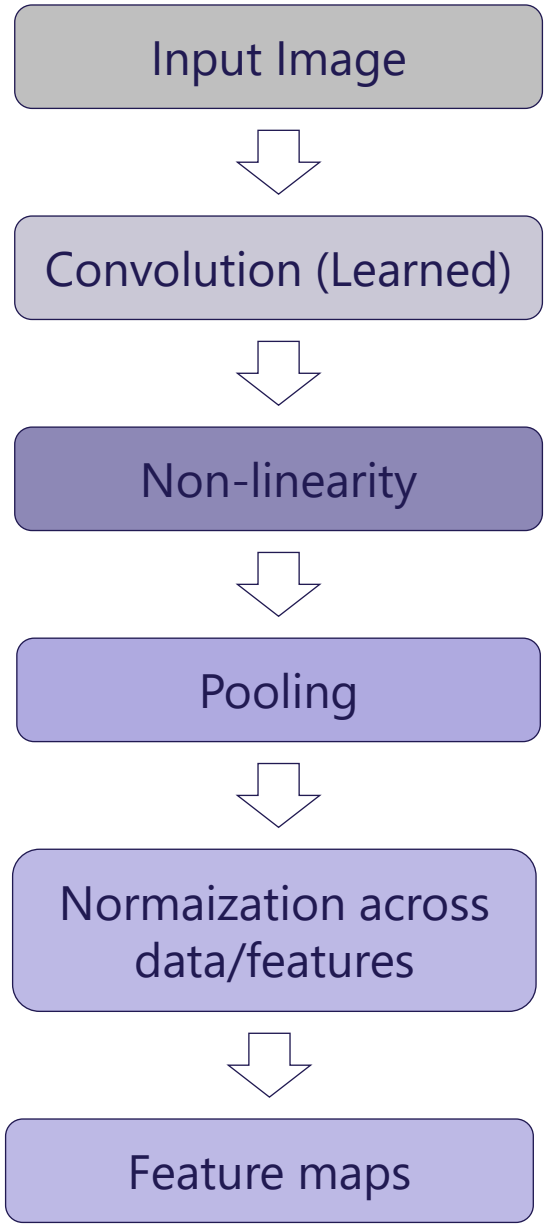
Batch normalization



Ioffe, Sergey, and Christian Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." *International Conference on Machine Learning*. 2015.

Summary of ConvNet (so far)

- Feed-forward:
  - Convolve input
  - Non-linearity (rectified linear)
  - [Optional] Pooling (local max)
  - [Optional] Batch Normalization
- Fully-connected classifier layer at top
- Supervised loss function (uses image label)
- Train convolutional filters by back-propagating classification error

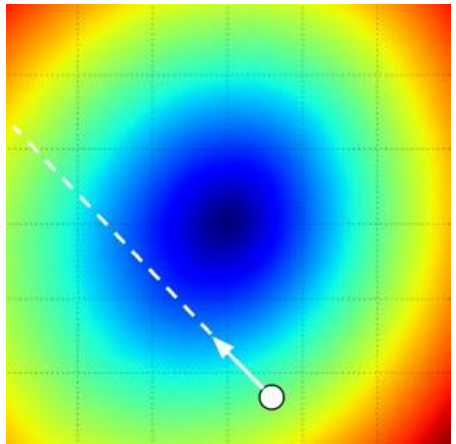


Optimization algorithms

- Gradient descent to optimize network weights and minimize loss.
- Error backpropagated through net using the chain rule.
- Mini-batch gradient decent (also referred to as Stochastic Gradient Descent (SGD)) to update, based on current batch D

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t; D)$$

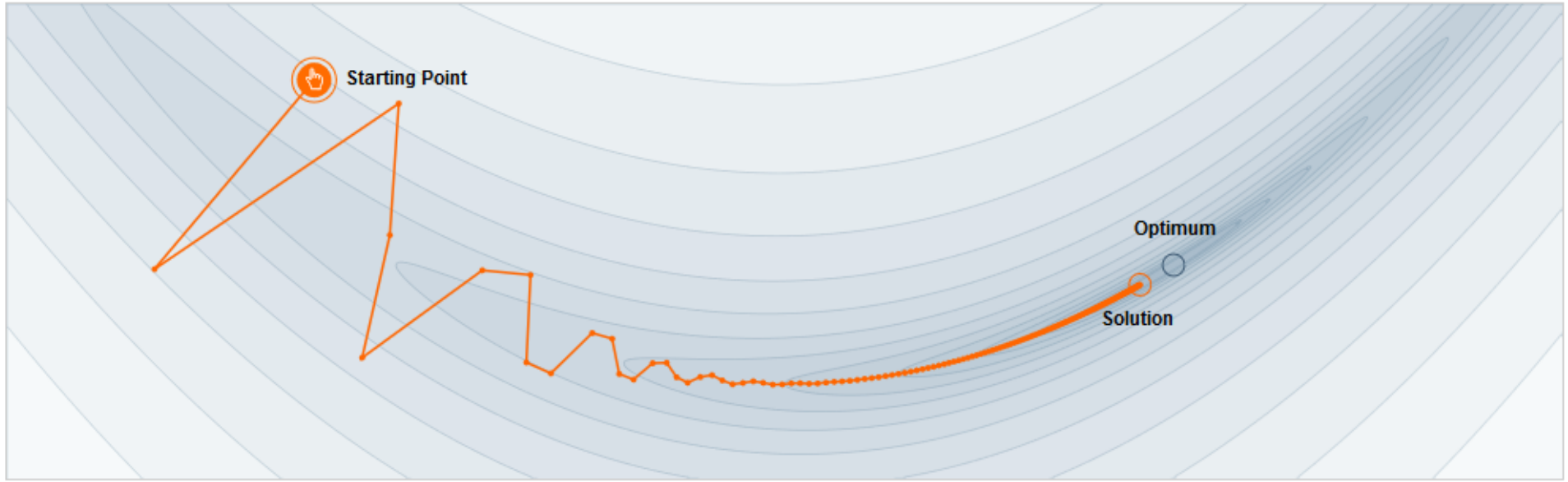
- Variants:
  - SGD + momentum
  - Nesterov accelerated gradient
  - Adagrad
  - Adadelta
  - RMSprop
  - Adaptive moment estimation (Adam)



Optimization algorithms

https://distill.pub/2017/momentum/

Why Momentum Really Works



Step-size α = 0.0040

0 0.003 0.006

Momentum β = 0.64

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

Architecture

- Which architecture?
- How deep (# layers)
- How wide? (size of convs)

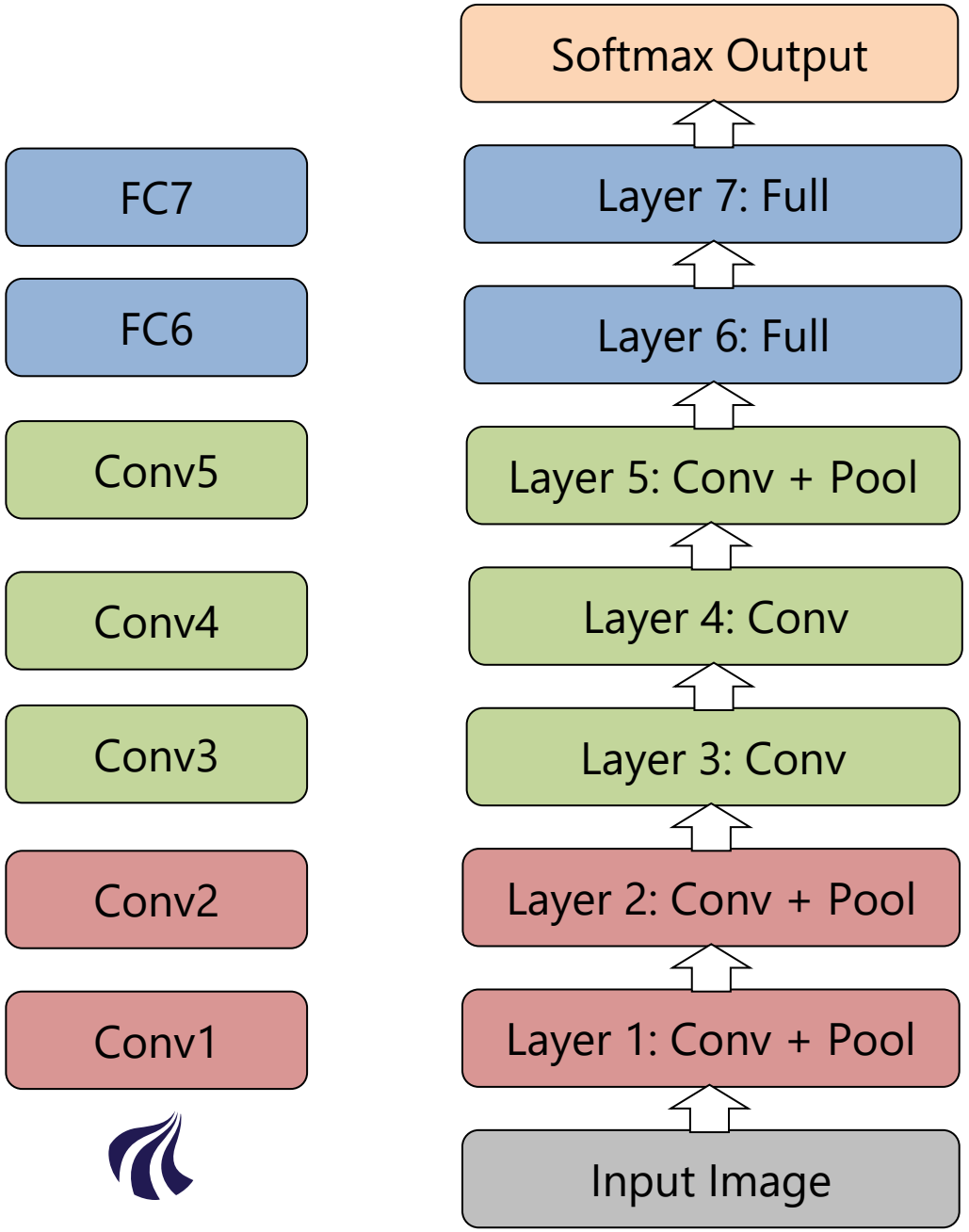
- Grid search
  - Massive amount of computations, often infeasible

- Deep learning
  - Learned features
  - Learned classification
  - **Handcrafted nets**

AlexNet architecture

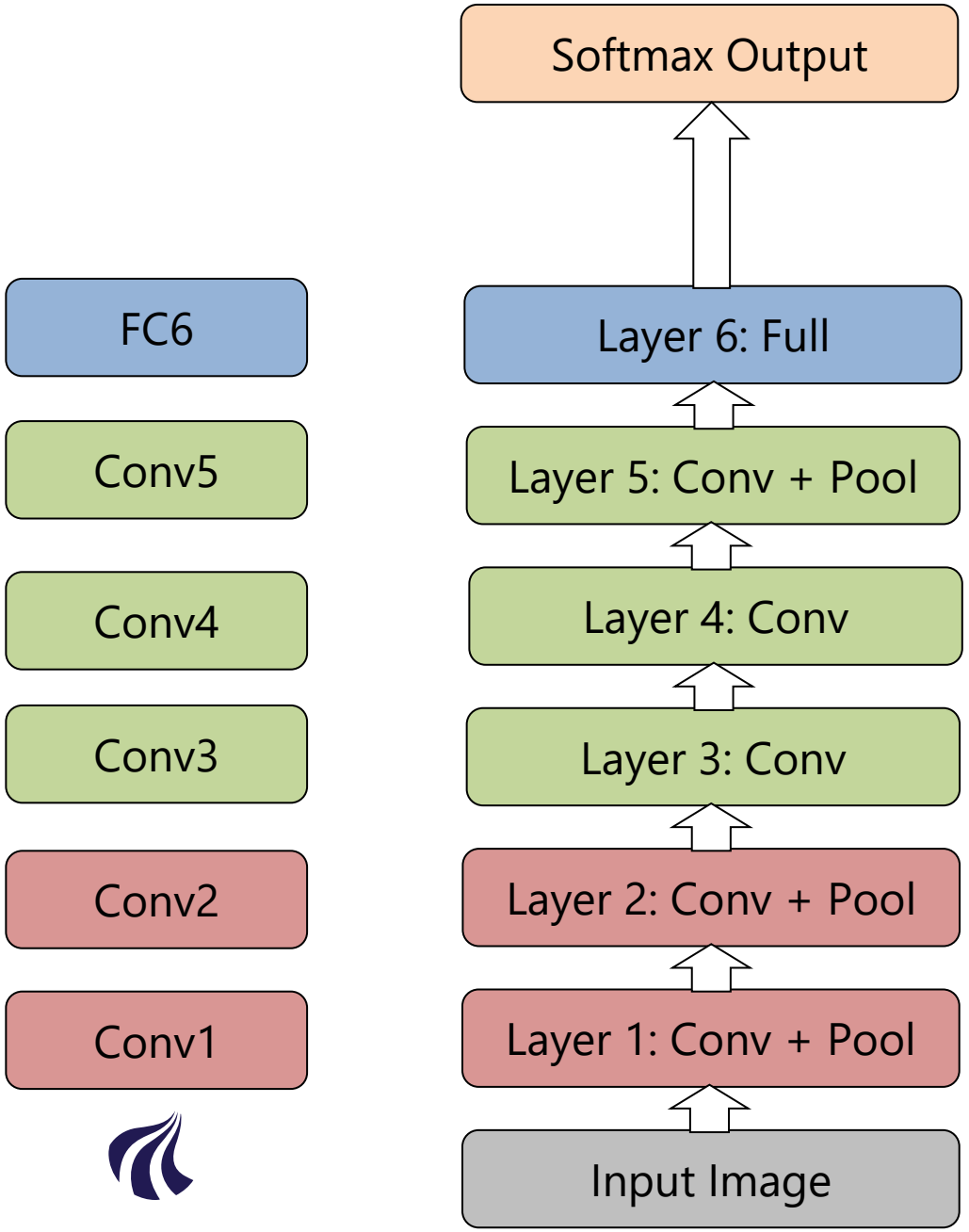
- 8 layers total
- 18,2 % top-5 error

- 60 million parameters



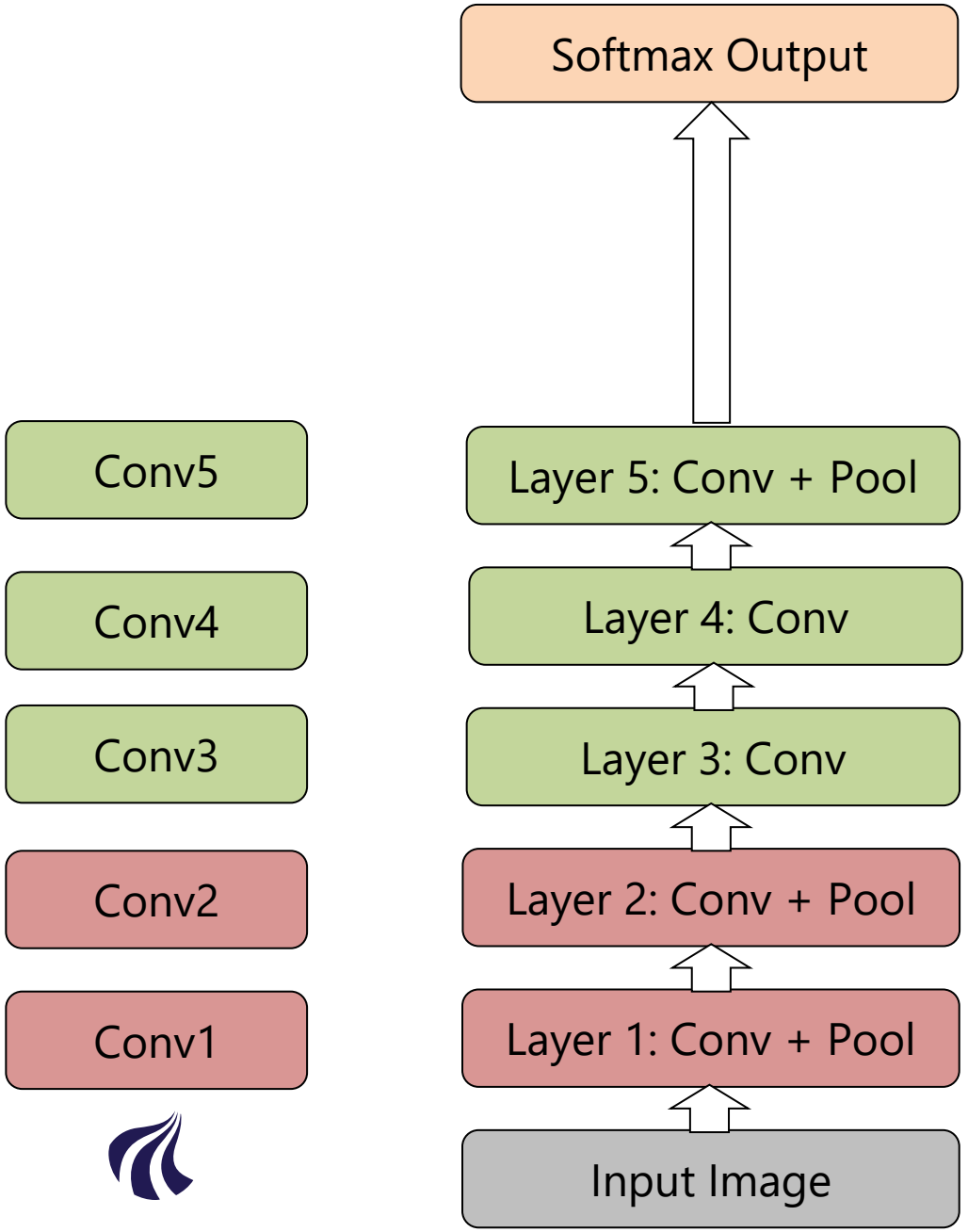
AlexNet architecture

- Remove:
  - FC7 fully connected layer
- Parameters:
  -16 million
- Performance:
  -1,1 %



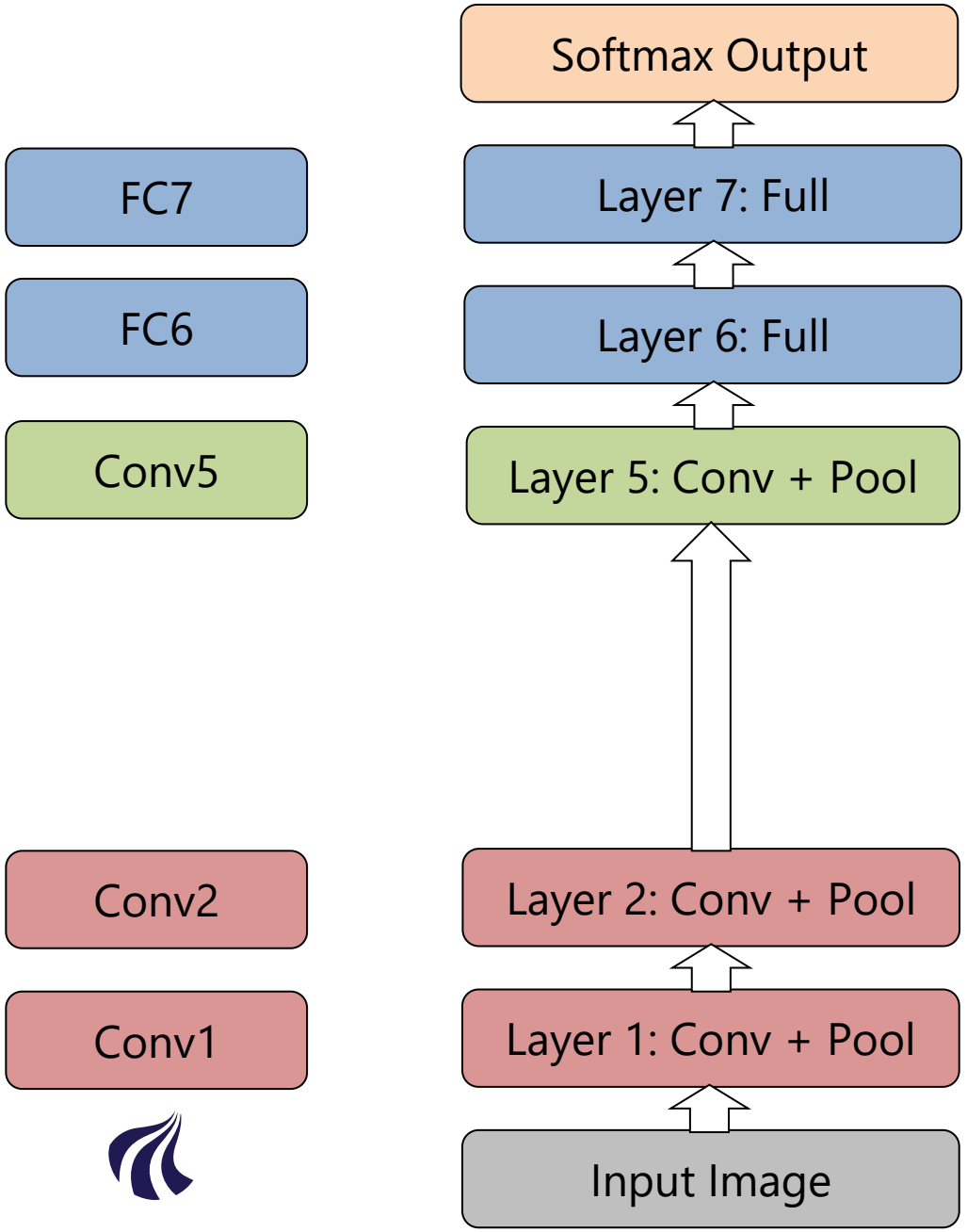
AlexNet architecture

- Remove:
  - FC6 fully connected layer
  - FC7 fully connected layer
- Parameters:
  -50 million
- Performance:
  -5,7 %



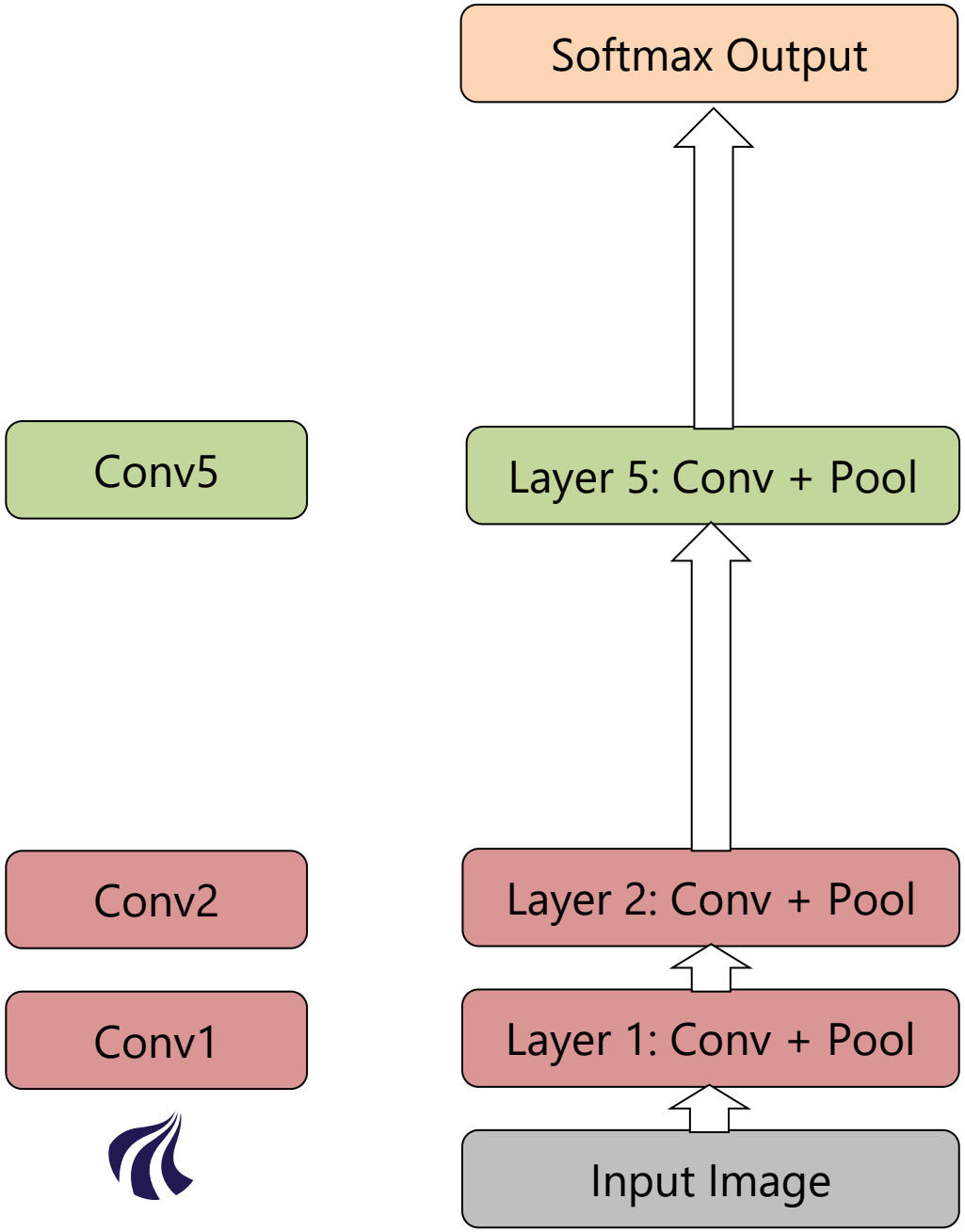
AlexNet architecture

- Remove: Upper feature extractor
  - Conv3 + pool3
  - Conv4 + pool3
- Parameters:
  -1 million
- Performance:
  -3,7 %



AlexNet architecture

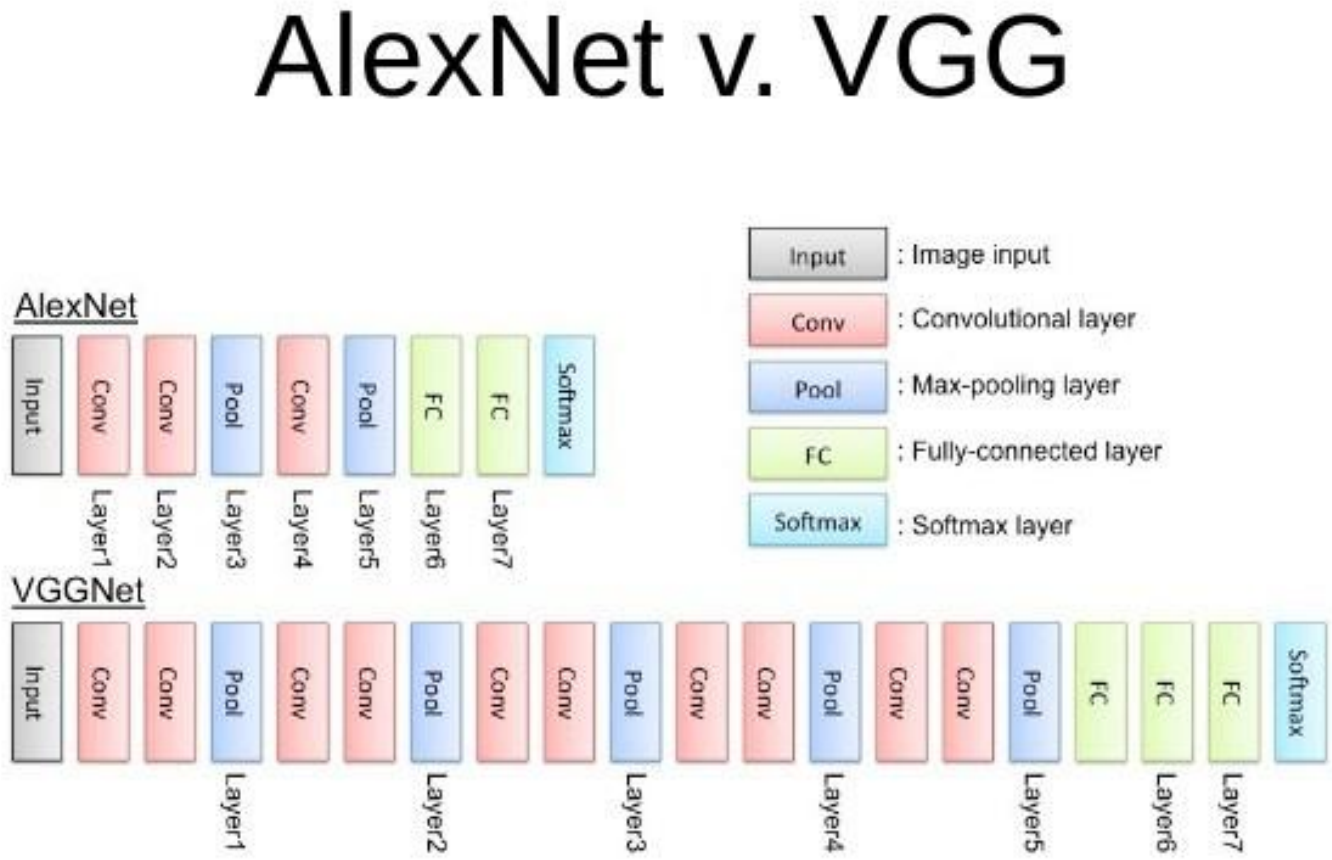
- Remove: Upper feature extractor
  - Conv3 + pool3
  - Conv4 + pool3
  - FC6 fully connected
  - FC7 fully connected
- Parameters:
  -51 million
- Performance:
  -33,5 %



VGG-Net (2015)

- Smaller filters (3x3)
  - Fewer parameters
  - Simulate larger receptive field when stacked

- Simonyan and Zisserman, ICLR, 2015



VGG-13

VGG-Net (2015)

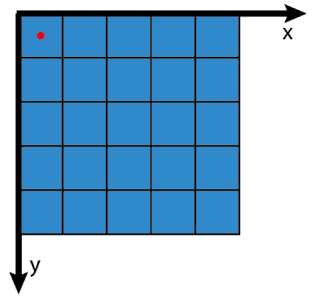
- Smaller filters (3x3)
  - Fewer parameters
  - Simulate larger receptive field when stacked

n : number of features
r : receptive field size
j : jump (distance between two consecutive features)
$start$: center coordinate of the first feature

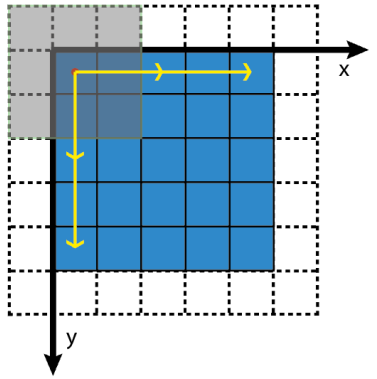
k : convolution kernel size
p : convolution padding size
s : convolution stride size

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$
$$j_{out} = j_{in} * s$$
$$r_{out} = r_{in} + (k-1) * j_{in}$$
$$start_{out} = start_{in} + \left(\frac{k-1}{2} - p\right) * j_{in}$$

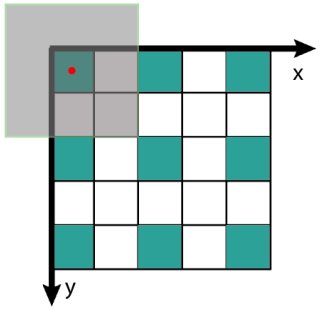
Layer 0: $n_0 = 5;\ r_0 = 1;\ j_0 = 1;$ $start_0 = 0.5$



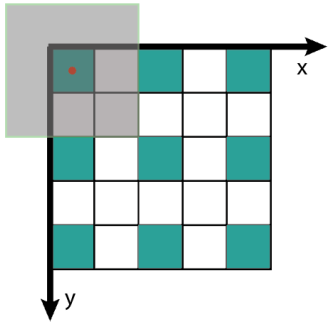
Conv1: $k_1 = 3;\ p_1 = 1;\ s_1 = 2$



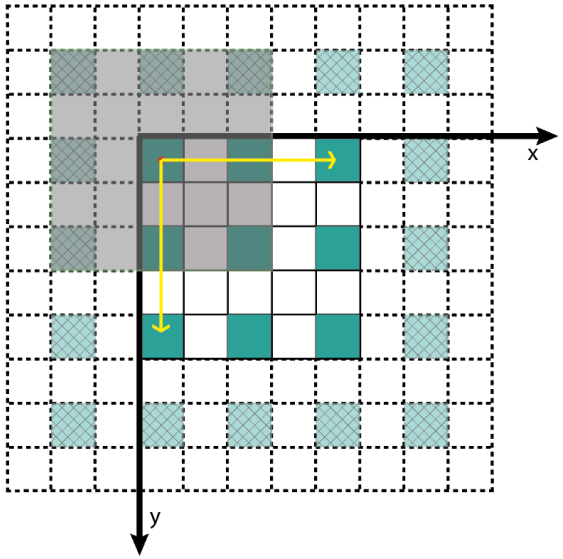
Layer 1: $n_1 = 3;\ r_1 = 3;\ j_1 = 2;$ $start_1 = 0.5$



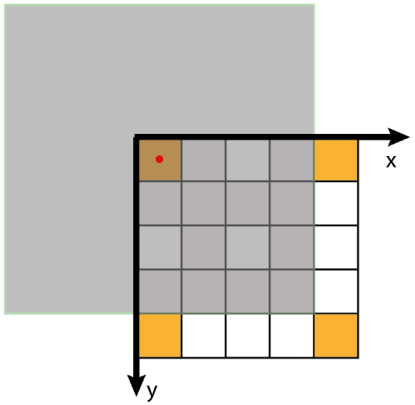
Layer 1: $n_1 = 3;\ r_1 = 3; j_1 = 2;$ $start_1 = 0.5$



Conv2: $k_2 = 3;\ p_2 = 1;\ s_2 = 2$



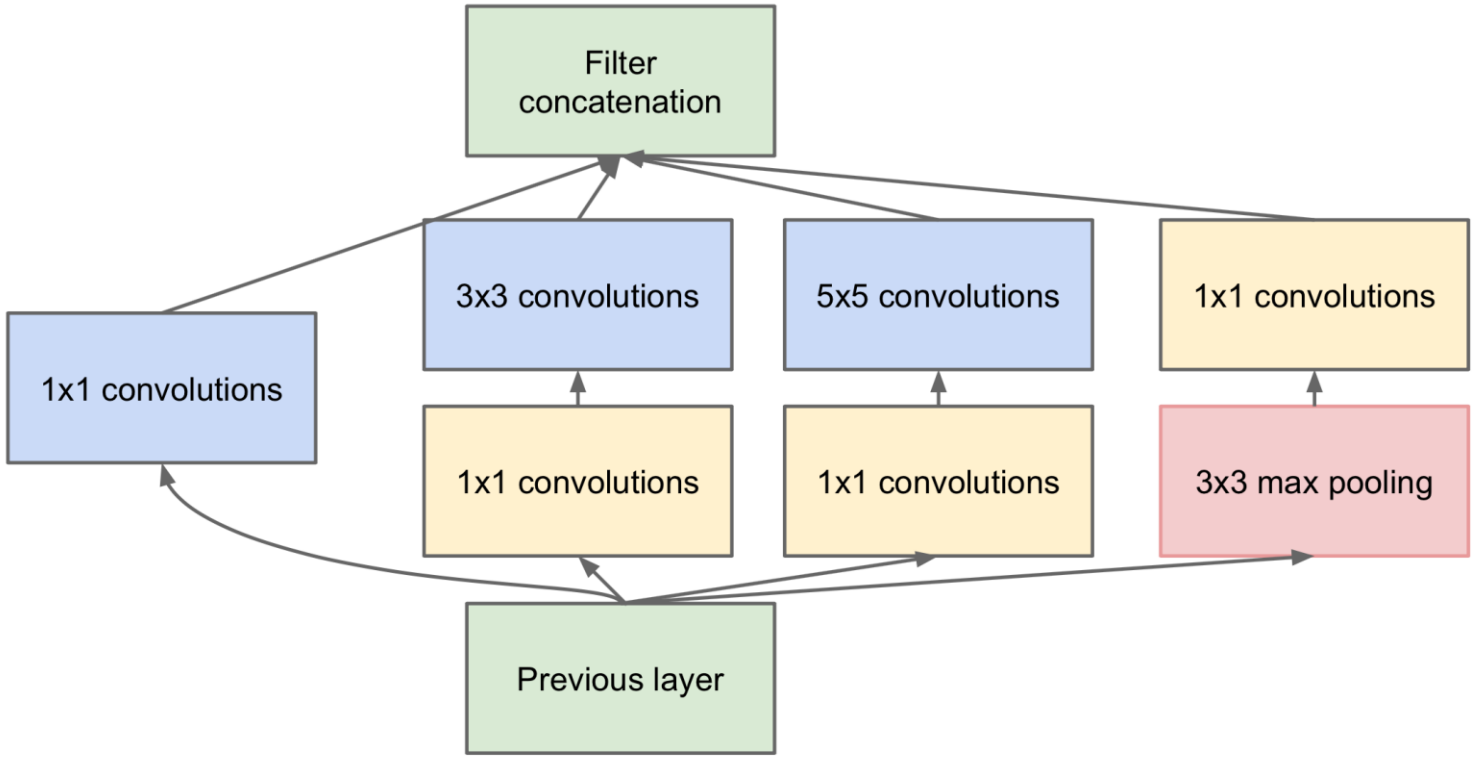
Layer 2: $n_3 = 2;\ r_3 = 7; j_3 = 4;$ $start_1 = 0.5$



https://medium.com/mlreview/a-guide-to-receptive-field-arithmetic-for-convolutional-neural-networks-e0f514068807

GoogLeNet (2015)

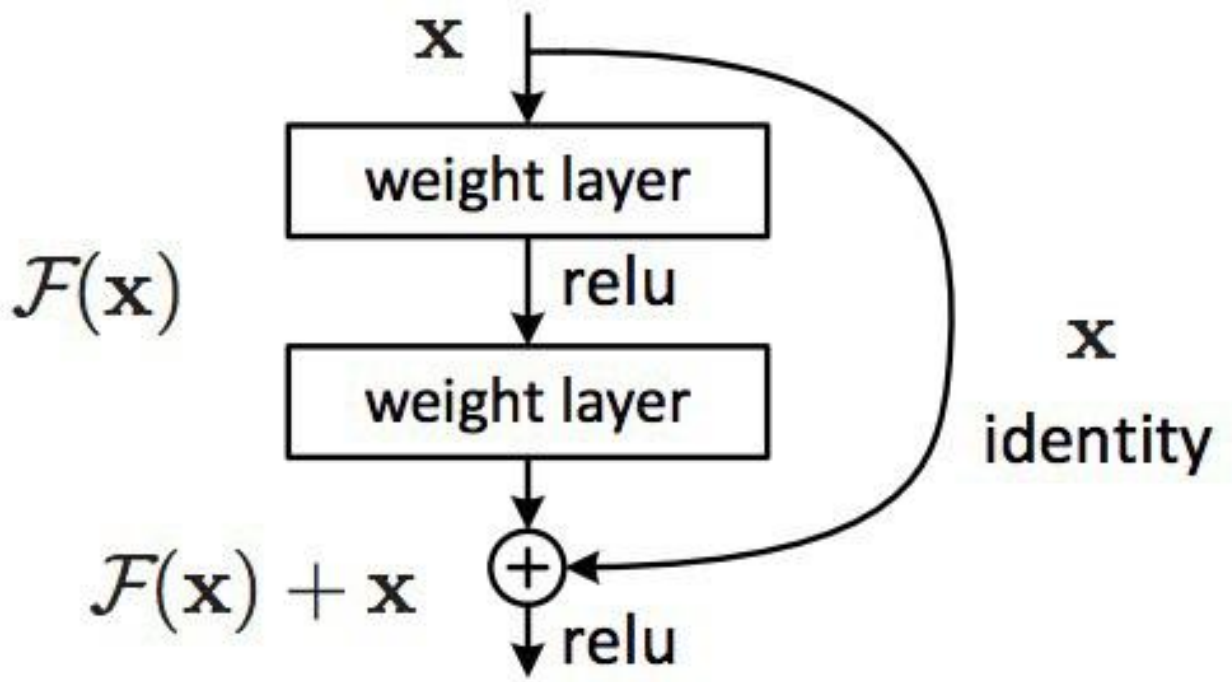
- Introducing ”inception” module
  - Parallel, different-size convolutions
- 22 layers, 6.7 M parameters
- No fully-connected layer, fewer parameters



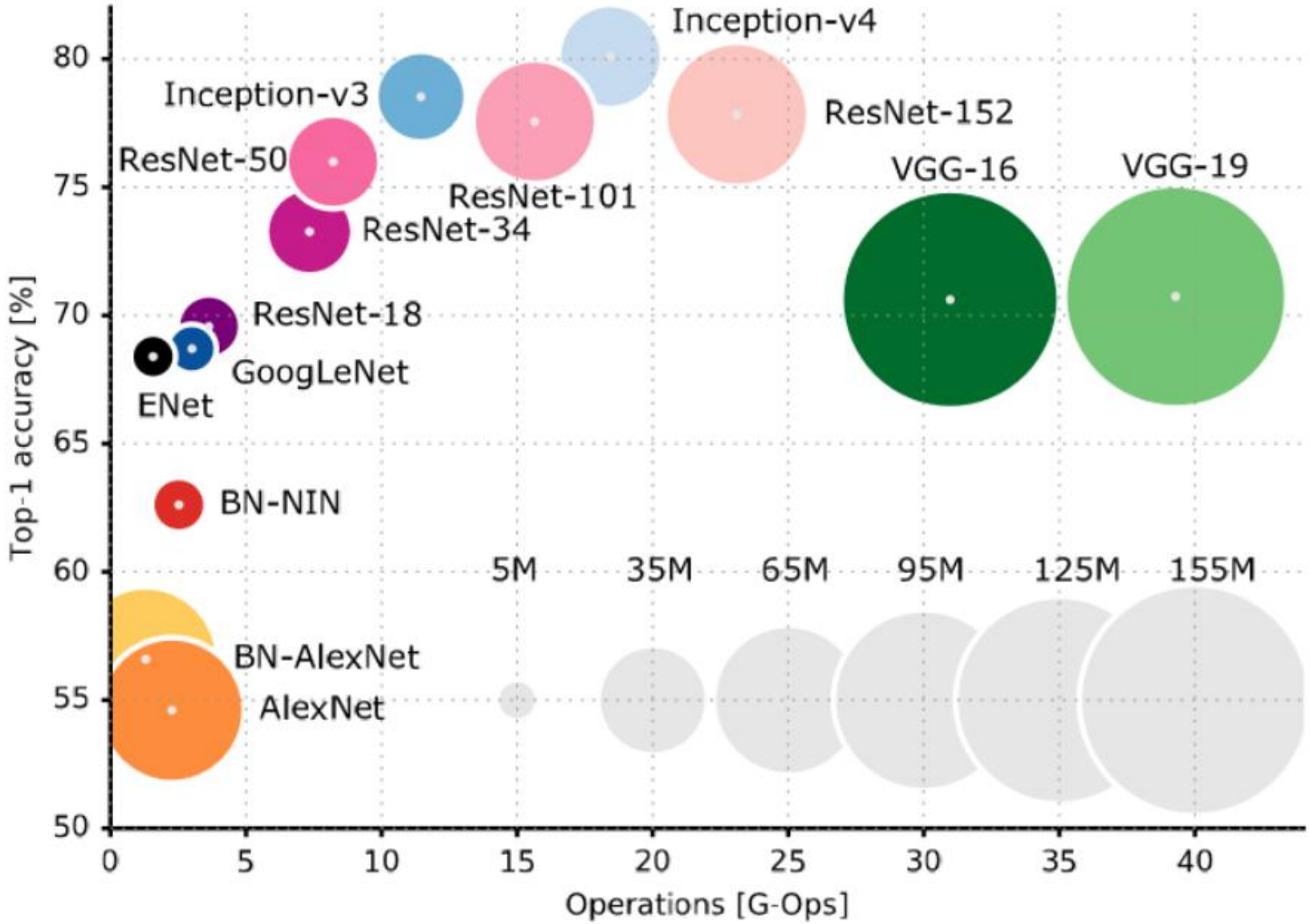
(b) Inception module with dimension reductions

ResNet (2016)

- More and more layers - hard to train, performance saturates
- Key idea: introducing skip connections
- Only residual needs to be learned
- Enables very deep networks
  - 152 layers, 60 M parameters
- Top-5 accuracy: 3.57 %



Accuracy vs FLOPS



Tips and Tricks



AALBORG UNIVERSITY
DENMARK

General network training tips

- Do a survey of which Neural Network architecture is most suitable for the required problem.
- Define Neural Network architecture
- Convert data to right format, shuffle, divide in batches
- Pre-process the data
- Augment Data
- Run neural network
- Train and monitor changes in training and validation accuracy.
- Test your model and save it for future use.



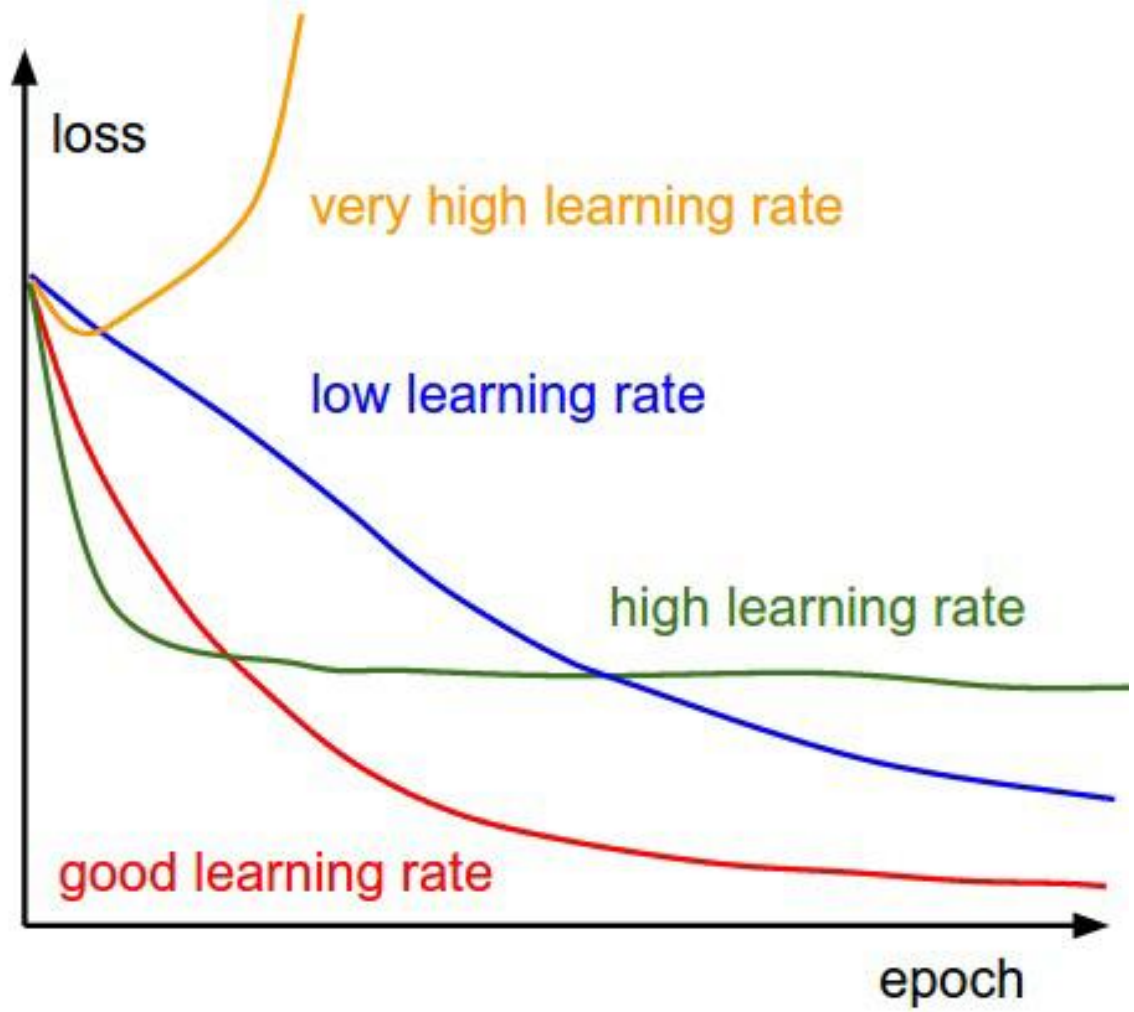
Data augmentation

- Make network more robust to translational changes, changes in lighting, etc.
- Techniques:
  - Cropping
  - Mirroring
  - Brightness
  - Color jittering
  - (Constrained) rotation



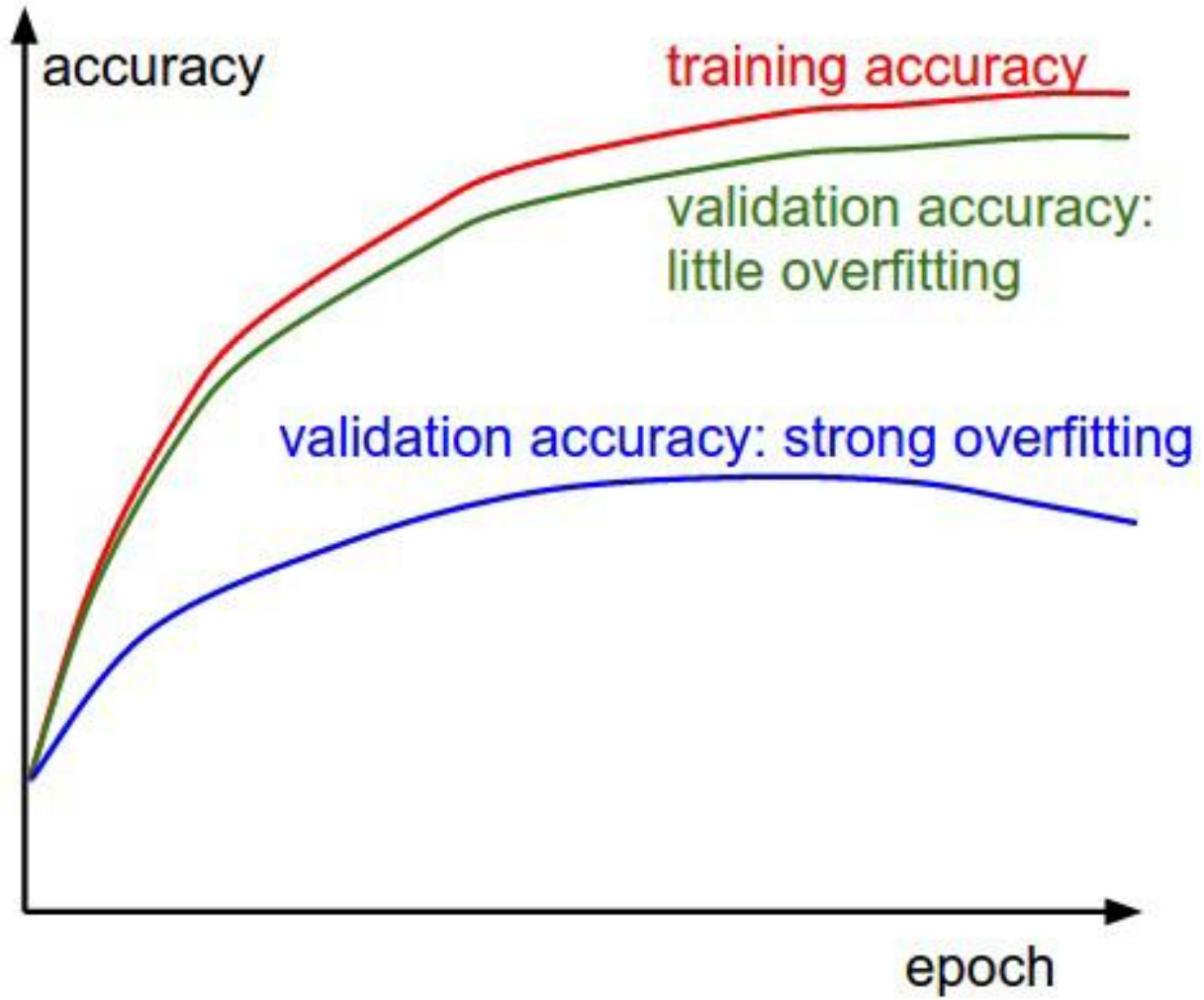
Monitoring training

- Train/validation data is typically split 70/30 (or 70/20/10).
- Tune base learning rate and learning rate decay to result in nice learning curve



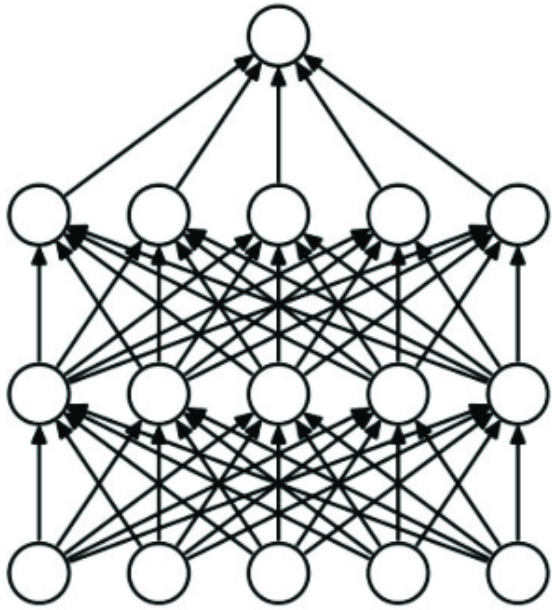
Monitoring training

- Train/validation data is typically split 70/30 (or 70/20/10).
- Tune base learning rate and learning rate decay to result in nice learning curve
- Important to monitor train and validation accuracy to avoid overfitting.

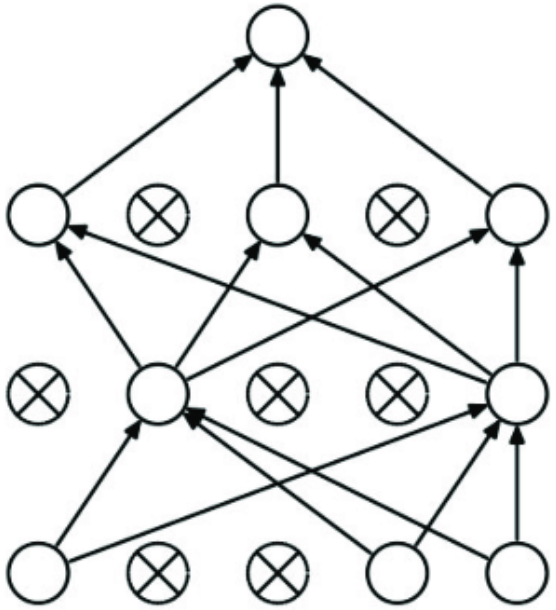


Regularization

- Avoid overfitting
  - Add L2 regularization (weight decay)
  - Add more training data
  - Use less complex models (not always best idea)
  - Add dropout



(a) Standard Neural Network



(b) Neural Net with Dropout

Frameworks

- Google or Facebook?
- Pick your evil ☺







References

- Slide 4-13, 36-38, 40-42, 44-54, 56-65
  Adapted from Rob Fergus,
  https://cs.nyu.edu/~fergus/teaching/vision/index.html
  (Who adapted some of the slides from elsewhere)

- Slide 14-17 from Andrej Karparthy's Stanford course notes: http://cs231n.github.io/

Schedule updates

- Next lecture: Thursday October 1st

- Depth sensing and camera-robot calibration will be taught in November
  - Depth sensing topic is now available as video lectures if you need it for your projects (lecture 10 on Moodle)

Thank you for listening!